

19/05/2025

Technical Report

Modern NLP

1. Introduction	2
2. Lab Implementation Summaries and Analysis	2
2.1 Session 1: Classical NLP Approaches	2
2.2 Session 2: Word Embeddings and Semantic Representations	5
2.3 Session 3: Transformers and Contextual Language Models	8
2.4 Session 4: Generative AI and Prompting Techniques	12
2.5 Session 5: Retrieval-Augmented Generation	13
3. Comprehensive Comparison	17
4. Conclusion and Future Work	18
5. References and Appendices	19

1. Introduction

- *Overview of modern NLP techniques and their progression*
- *Brief summary of the labs completed and their objectives*
- *Outline of your technical approach and implementation choices*

This report presents the work we carried out as part of the Modern NLP (natural language processing) course. Through five practical sessions, we gradually discovered and experimented different approaches used today in NLP, from classical methods to the most advanced models.

Each session had a very specific objective. We started by building a text classification pipeline using classic techniques such as Bag-of-Words and TF-IDF. Next, we studied word embeddings such as Word2Vec and GloVe, which gave us a better understanding of the semantic relationships between words. In the third part, we worked with transformers and learned how to fine-tune a BERT model. The fourth session focused on generative models (such as GPT) and the art of prompt engineering. Finally, we set up a Retrieval-Augmented Generation (RAG) system, which combines information retrieval and text generation.

This report summarises the various stages we followed, our technical choices, the results obtained and our analyses. Each section corresponds to one of the 5 sessions of the course. We have also included concrete examples, screenshots and code extracts to clearly illustrate our work.

2. Lab Implementation Summaries and Analysis

2.1 Session 1: Classical NLP Approaches

- *Summary of your implementation of the text classification pipeline*
- *Analysis of feature extraction methods (bag-of-words, TF-IDF)*
- *Error analysis detailing the limitations of classical models*
- *Code snippets or pseudo-code highlighting critical components*

In this first session, the main objective was to build a complete text classification pipeline using classic NLP techniques, before the advent of deep learning models. We used the 'SMS Spam Collection' dataset, which contains SMS messages labelled as 'spam' or 'ham' (non-spam), to train models capable of detecting spam.

The first step was to clean and pre-process the texts. We converted all messages to lower case, removed punctuation, special characters and sometimes stop words. We also tested stemming, which consists of reducing words to their root (for example, 'running' becomes 'run'), to standardise the vocabulary.

```
# Preprocessing function
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Tokenize
    tokens = word_tokenize(text)
    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [w for w in tokens if w not in stop_words]
    # Apply Stemming
    stemmer = PorterStemmer()
    stemmed = [stemmer.stem(w) for w in filtered_tokens]
    return ' '.join(stemmed)
```

We then converted the messages into numerical vectors using two methods: Bag-of-Words (which counts the occurrences of each word) and TF-IDF (which gives more importance to rare words). We also tried n-grams, which capture groups of words such as 'not good', useful for detecting certain expressions such as negations.

```
# Bag of Words
bow_vectorizer = CountVectorizer(max_features=5000)
X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)

# TF-IDF
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

Once the texts had been transformed, we trained several classic machine learning models:

- Naive Bayes: fast, simple and effective for this type of task.
- Logistic Regression: more robust than Naive Bayes in certain cases.

```
# Naive Bayes
nb = MultinomialNB()
nb.fit(X_train_bow, y_train)

# Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_tfidf, y_train)

# Hyperparameter tuning
param_grid = {'C': [0.1, 1, 10, 100]}
grid = GridSearchCV(LogisticRegression(), param_grid, cv=5)
grid.fit(X_train_tfidf, y_train)
print(f'Best parameters: {grid.best_params_}')
```

Best parameters: {'C': 100}

Naive Bayes Accuracy: 0.9839
Logistic Regression Accuracy: 0.9677
Naive Bayes Report:

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	955
spam	0.97	0.92	0.94	160
accuracy			0.98	1115
macro avg	0.98	0.96	0.97	1115
weighted avg	0.98	0.98	0.98	1115

The models were evaluated using classic metrics such as accuracy, F1-score, precision and recall.

Overall, performance was good, but we quickly noticed certain limitations:

- The BoW or TF-IDF models do not understand the meaning of words, nor their order.
- They are sensitive to words that are absent during training (fixed vocabulary problem).
- They do not handle sentences containing negations well ('not bad' can be misclassified as negative because the word 'bad' dominates the signal).
- No overall context is taken into account (each word is treated independently).

```
# Find misclassified examples
mistakes = np.where(y_pred_lr != np.array(y_test))[0]
for idx in mistakes[:10]: # Show first 10 mistakes
    print(f'Text: {X_test.iloc[idx]}')
    print(f'Actual: {y_test.iloc[idx]}')
    print(f'Predicted: {y_pred_lr[idx]}')
    print('----')

# Feature importance (Logistic Regression)
feature_names = tfidf_vectorizer.get_feature_names_out()
coef = lr.coef_[0]

top_pos = np.argsort(coef)[-10:] # Top 10 positive
top_neg = np.argsort(coef)[:10] # Top 10 negative

print('Top positive features:', [feature_names[i] for i in top_pos])
print('Top negative features:', [feature_names[i] for i in top_neg])
```

We also carried out an error analysis, looking at some misclassified examples. We found that many of the errors were related to message ambiguity or lack of context, which is to be expected with such simple models.

```
Text: ringtoneking 84484
Actual: spam
Predicted: ham
---
Text: Can U get 2 phone NOW? I wanna chat 2 set up meet Call me NOW on 09896182316 U can cum here 2moro Luv JANE xx Calls&#160;/minmoremobsEH5P0Box45P0139WA
Actual: spam
Predicted: ham
---
Text: Ever thought about living a good life with a perfect partner? Just txt back NAME and AGE to join the mobile community. (180p/SMS)
Actual: spam
Predicted: ham
---
Text: I want some cock! My hubby's away, I need a real man 2 satisfy me. Txt WIFE to 89938 for no strings action. (Txt STOP 2 end, txt rec Â£1.50ea. 01Box 731 LA1 7WS. )
Actual: spam
Predicted: ham
---
Text: 08714712388 between 10am-7pm Cost 10p
Actual: spam
Predicted: ham
---
Text: Ur balance is now Â£600. Next question: Complete the landmark, Big, A. Bob, B. Barry or C. Ben ?. Text A, B or C to 83738. Good luck!
Actual: spam
Predicted: ham
---
Text: I (Career Tel) have added u as a contact on INDYAROCKS.COM to send FREE SMS. To remove from phonebook - sms NO to 611&#62;
---
Predicted: ham
---
Top positive features: ['reply', 'uk', 'claim', 'www', 'mobile', 'stop', 'text', 'free', 'call', 'txt']
Top negative features: ['me', 'my', 'ok', 'that', 'it', 'gt', 'lt', 'later', 'but', 'can']
```

Finally, we were able to use a pre-trained model to see if it performed better on the same given task. The aim was to see how recent models handle complex cases.

```
# Example code (if time permits)
from transformers import pipeline

# Load pretrained classifier
classifier = pipeline('sentiment-analysis')

# Test on error cases
for idx in mistakes[:5]: # First 5 mistakes
    text = X_test.iloc[idx]
    modern_result = classifier(text)[0]
    print(f'Text: {text}')
    print(f'Classical prediction: {y_pred_lr[idx]}')
    print(f'Modern prediction: {modern_result["label"]}')
    print(f'Confidence: {modern_result["score"]:.4f}')
    print(f'Actual label: {y_test.iloc[idx]}')
    print('----')
```

```
Text: ringtoneking 84484
Classical prediction: ham
Modern prediction: NEGATIVE
Confidence: 0.7169
Actual label: spam
----
Text: Can U get 2 phone NOW? I wanna chat 2 set up meet Call me NOW on 09096102316 U can cum here 2moro Luv JANE
Classical prediction: ham
Modern prediction: NEGATIVE
Confidence: 0.9946
Actual label: spam
----
Text: Ever thought about living a good life with a perfect partner? Just txt back NAME and AGE to join the mobil
Classical prediction: ham
Modern prediction: NEGATIVE
Confidence: 0.9134
Actual label: spam
----
```

To sum up, this tutorial enabled us to discover the basics of traditional NLP and its limitations. It also served as a starting point for understanding why more modern approaches, such as word embeddings or transformers, have become necessary.

2.2 Session 2: Word Embeddings and Semantic Representations

- *Document your exploration of pretrained embeddings*
- *Analysis of word similarity and analogies experiments*
- *Results from your document clustering exercise with visualizations*
- *Discussion of how embeddings improved upon classical approaches*

In this tutorial, we explored the concept of word embeddings, a method for representing words in the form of dense numerical vectors. Unlike traditional approaches such as Bag-of-Words, these vectors take into account the semantic relationships between words. For example, words like 'holiday' and 'vacation' will have vectors that are close because they are often used in similar contexts.

We began by loading pre-trained GloVe vectors using the gensim library. We then searched for words most similar to 'holiday'. The model returned words such as 'christmas', 'thanksgiving', 'easter' and 'vacation', with a high similarity score (close to 1), showing that the vectors capture meaning relationships well.

```
# Load pretrained GloVe vectors
import gensim.downloader

# Load GloVe embeddings (smaller version for quick loading)
glove_vectors = gensim.downloader.load('glove-wiki-gigaword-100')

# Find most similar words
similar_to_holiday = glove_vectors.most_similar('holiday', topn=10)
print('Words most similar to "holiday":')
for word, similarity in similar_to_holiday:
    print(f'{word}: {similarity:.4f}')
```

```
Words most similar to "holiday":
christmas: 0.8737
holidays: 0.8367
thanksgiving: 0.8075
easter: 0.7502
vacation: 0.7092
day: 0.6919
festive: 0.6855
weekend: 0.6793
celebration: 0.6680
halloween: 0.6652
```

We then tested known semantic analogies with GloVe vectors. For example, if we do 'king - man + woman', the main result is 'queen', which is expected. We also tested a geographical relationship: 'paris - france + italy', which gave 'rome', 'milan', and so on. These tests show that GloVe vectors succeed in capturing complex relationships between words.

```
# Test vector analogies
# Using the same glove_vectors from Task 1
# Classic king - man + woman analogy
result = glove_vectors.most_similar(
    positive=[ 'woman', 'king'],
    negative=[ 'man' ],
    topn=5
)

print( 'king - man + woman = ?' )
for word, similarity in result:
    print(f'{word}: {similarity:.4f}')

# Try a capital-country analogy
result = glove_vectors.most_similar(
    positive=[ 'paris', 'italy'],
    negative=[ 'france'],
    topn=5
)

print('\nparis - france + italy = ?')
for word, similarity in result:
    print(f'{word}: {similarity:.4f}')
```

```
king - man + woman = ?
queen: 0.7699
monarch: 0.6843
throne: 0.6756
daughter: 0.6595
princess: 0.6521

paris - france + italy = ?
rome: 0.8190
milan: 0.7376
naples: 0.7118
venice: 0.7015
turin: 0.6995
```

To take this a step further, we used the SentenceTransformer model to obtain embeddings of whole sentences. We encoded three simple sentences ('The cat sits on the mat', 'A dog is lying on the rug', etc.), then compared their similarity using cosine distance. The results show that the first two sentences (about animals on a mat) are closer together than the third, confirming that the model does indeed capture the overall meaning of the sentences.

```
# Load a pretrained sentence embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Example sentences
sentences = [
    'The cat sits on the mat',
    'A dog is lying on the rug',
    'I went to the market today'
]
sentences
```

```
['The cat sits on the mat',
 'A dog is lying on the rug',
 'I went to the market today']
```

embeddings

```
array([[ 0.13489066, -0.03206339, -0.02033526, ...,  0.04674225,
         0.0149245 ,  0.0586335 ],
       [-0.00422229,  0.0249036 ,  0.02663611, ...,  0.01858228,
         0.02658677,  0.05017022],
       [-0.02142073, -0.0111344 , -0.02078409, ..., -0.10816024,
        -0.09534409,  0.06974486]], dtype=float32)
```

```
# Calculate similarity matrix
similarity_matrix = cosine_similarity(embeddings)
similarity_matrix
```

```
array([[1.          ,  0.42119998,  0.01949149],
       [0.42119998,  0.99999999,  0.04020148],
       [0.01949149,  0.04020148,  1.          ]], dtype=float32)
```

```
# Print similarity scores
for i in range(len (sentences) ):
    for j in range(i + 1, len(sentences)):
        print(f'Similarity between "{sentences[i]}" and "{sentences[j]}": {similarity_matrix[i][j]:.4f}')

Similarity between "The cat sits on the mat" and"A dog is lying on the rug": 0.4212
Similarity between "The cat sits on the mat" and"I went to the market today": 0.0195
Similarity between "A dog is lying on the rug" and"I went to the market today": 0.0402
```

Finally, we performed unsupervised clustering on a dataset of restaurant reviews. We transformed each review into a vector using Sentence-BERT, then used the KMeans algorithm to group the reviews into 5 clusters. When we analysed the clusters, we noticed that some groups were made up of negative reviews of the food, others of customer service, and so on. This shows that embeddings can be used to automatically group texts by theme.

```
# Load example dataset (from provided files or download)
# This example uses a dataset of customer reviews
df = pd. read_csv('Restaurant_Reviews.tsv', sep='\t')
texts = df['Review']. tolist()

# Create embeddings (using the model from Task 3)
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(texts)

# Perform K-means clustering
n_clusters = 5 # Adjust based on your data
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
cluster_labels = kmeans.fit_predict(embeddings)

# Add cluster labels to dataframe
df['cluster'] = cluster_labels

# Examine clusters
for cluster_id in range(n_clusters):
    cluster_texts = df[df['cluster'] == cluster_id]['Review']
    print(f'Cluster {cluster_id} ({len(cluster_texts)}reviews):')
    # Print 5 examples from this cluster
    for text in cluster_texts. head (10):
        print(f'- {text[: 100]}...')
    print()
```

```
Cluster 0 (204reviews):
- Not tasty and the texture was just nasty....
- Honestly it didn't taste THAT fresh....
- The potatoes were like rubber and you could tell they had been made up ahead of time being kept unde...
- The fries were great too....
- - They never brought a salad we asked for...
- Took an hour to get our food only 4 tables in restaurant my food was Luke warm, Our sever was runnin...
- The worst was the salmon sashimi....
- The only redeeming quality of the restaurant was that it was very inexpensive....
- Hard to judge whether these sides were good because we were grossed out by the melted styrofoam and ...
- Frozen pucks of disgust, with some of the worst people behind the register....

Cluster 1 (192reviews):
- Now I am getting angry and I want my damn pho....
- A great touch....
- Would not go back....
- I was disgusted because I was pretty sure that was human hair....
- I was shocked because no signs indicate cash only....
- did not like at all....
- So they performed....
- This was like the final blow!...
- There is not a deal good enough that would drag me into that establishment again....
- He came running after us when he realized my husband had left his sunglasses on the table....

Cluster 2 (275reviews):
...
- They have horrible attitudes towards customers, and talk down to each one when customers don't enjoy...
- The service was meh....
- 2 times - Very Bad Customer Service !...
```

To visualise the clustering results more intuitively, we used the t-SNE algorithm to reduce the size of the document vectors to two dimensions. We then displayed the points on a graph, with a different colour for each cluster.

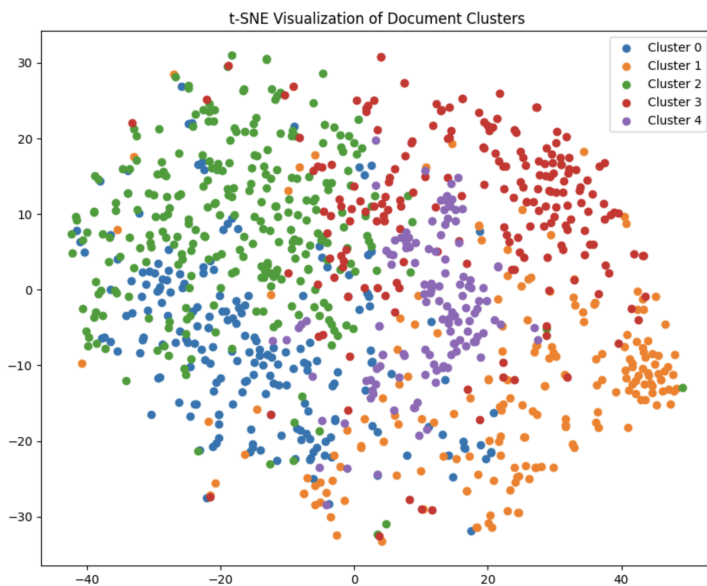
```
# Visualizing document embeddings in 2D
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Using embeddings and cluster_labels from Task 4
# Apply t-SNE for dimensionality reduction
tsne = TSNE(n_components=2, random_state=42 )
embeddings_2d = tsne.fit_transform(embeddings)

# Create scatter plot
plt.figure(figsize=(10, 8))
```

```
# Plot each cluster with a different color
for cluster_id in range(n_clusters):
    # Get points in this cluster
    cluster_points = embeddings_2d[cluster_labels == cluster_id]
    # Plot points with label
    plt.scatter (
        cluster_points[:, 0],
        cluster_points[:, 1],
        label=f'Cluster {cluster_id}'
    )

plt.title('t-SNE Visualization of Document Clusters')
plt.legend()
plt.show()
```



The generated graph clearly shows that some clusters are well separated, while others are partially overlapping. This reflects the nature of the language: certain categories of comments may share similar themes or expressions. This type of visualisation is very useful for interpreting the semantic structure of a set of texts.

This tutorial gave us a better understanding of why distributed representations have revolutionised NLP. Unlike traditional approaches, they allow semantic searches, thematic groupings and even analogical reasoning. We also saw that these representations are an essential starting point for modern models such as BERT or GPT.

2.3 Session 3: Transformers and Contextual Language Models

- *Details of your fine-tuning process for a transformer model*
- *Comparative analysis between transformer performance and classical models*
- *Error analysis demonstrating the contextual understanding advantages*
- *Discussion of specific cases where transformers excelled*

This session was dedicated to discovering transformers, an architecture that has completely changed language processing since 2017. Unlike traditional approaches or static word embeddings (seen in previous sessions), transformers generate contextual representations of words: a word like 'bank' will not have the same vector depending on whether we're talking about money or a river. This operation is based on the self-attention mechanism, which enables the model to understand the relationships between all the words in a sentence, whatever their position.

In the tutorial, we worked on sentiment classification using film reviews from the IMDb dataset. The aim was to fine-tune a BERT model (pre-trained by Hugging Face) on this dataset.

```
# Load dataset using Hugging Face datasets
imdb = load_dataset('imdb')
# Examine dataset structure
print(imdb)
# Load tokenizer for BERT
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
```

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

We first loaded the dataset and explored its structure: 25,000 reviews for training, 25,000 for testing, and 50,000 unlabelled.

```
def tokenize_function (examples):
    # Tokenize all texts and truncate to max length of 512
    return tokenizer(examples['text'], padding='max_length', truncation=True)

# Apply tokenization to whole dataset
tokenized_imdb = imdb.map(tokenize_function, batched=True)

# Convert to PyTorch tensors and create dataloaders
tokenized_imdb = tokenized_imdb.remove_columns(['text'])
tokenized_imdb = tokenized_imdb.rename_column('label', 'labels')
tokenized_imdb.set_format('torch')
```

We then applied a BERT tokeniser to transform each critic into a sequence of tokens (with padding and truncation). The dataset was converted to PyTorch format for use with the model.

```
# Load BERT with classification head
from transformers import AutoModelForSequenceClassification
```

```
model = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2) # Binary classification: positive/negative
```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: [You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
training_args = TrainingArguments(output_dir='./results', per_device_train_batch_size=4, per_device_eval_batch_size=16, num_train_epochs=3, weight_decay=0.01,
trainer = Trainer(model=model, args=training_args, train_dataset=tokenized_imdb['train'], eval_dataset=tokenized_imdb['test'])

# Start fine-tuning
trainer.train()
```

As we can see above, the model used was bert-base-uncased, with a binary classification head (positive or negative).

The fine-tuning stage was prepared using Hugging Face's Trainer class.

Unfortunately, fully training the model takes too long on a conventional machine without a powerful GPU, so this part could not be completed.

It takes too much time so we will test BERT directly on the sentences.

```
# Charger le tokenizer et le modèle pré-entraîné
tokenizer = AutoTokenizer.from_pretrained("textattack/bert-base-uncased-imdb")
model = AutoModelForSequenceClassification.from_pretrained("textattack/bert-base-uncased-imdb")

# Passer en mode évaluation
model.eval()

# Phrase à tester
sentences = [
    "This movie was absolutely fantastic!",
    "I really hated this film. It was boring and too long.",
    "It was okay, not great, not terrible."
]

# Tokenizer les phrases
inputs = tokenizer(sentences, padding=True, truncation=True, return_tensors="pt")

# Faire les prédictions
with torch.no_grad():
    outputs = model(**inputs)
    predictions = torch.argmax(outputs.logits, dim=-1)

# Afficher les résultats
for sentence, prediction in zip(sentences, predictions):
    label = "Positive" if prediction.item() == 1 else "Negative"
    print(f"Sentence: {sentence}\nPrediction: {label}\n")
```

Sentence: This movie was absolutely fantastic!
Prediction: Positive

Sentence: I really hated this film. It was boring and too long.
Prediction: Negative

Sentence: It was okay, not great, not terrible.
Prediction: Positive

To get around this, we used a model already fine-tuned on IMDb (textattack / bert-base-uncased-imdb) and directly tested its ability to predict the sentiment of individual sentences. The model recognised emotions well: for example, it correctly classified 'This movie was absolutely fantastic!' as positive, and 'I really hated this film...' as negative.

We also tested SentenceTransformer to generate sentence embeddings from BERT, which can then be used to compare the semantic similarity between sentences.

We can use SentenceTransformer which is useful to obtain vector representations of sentences.

```
from sentence_transformers import SentenceTransformer, util

# Charger un modèle BERT optimisé pour les embeddings
model = SentenceTransformer('all-MiniLM-L6-v2')

# Tes phrases
sentences = [
    "This movie was absolutely fantastic!",
    "I really hated this film. It was boring and too long.",
    "It was okay, not great, not terrible."
]

# Obtenir les embeddings
embeddings = model.encode(sentences, convert_to_tensor=True)

# Afficher les vecteurs (optionnel)
for sentence, embedding in zip(sentences, embeddings):
    print(f"Sentence: {sentence}\nEmbedding shape: {embedding.shape}\n")
```

Sentence: This movie was absolutely fantastic!
Embedding shape: torch.Size([384])

Sentence: I really hated this film. It was boring and too long.
Embedding shape: torch.Size([384])

Sentence: It was okay, not great, not terrible.
Embedding shape: torch.Size([384])

For example, the similarity between two sentences of opposite polarity (positive vs. negative) remained low, showing that the vectors capture the nuance of meaning well.

```
cos_sim = util.pytorch_cos_sim(embeddings[0], embeddings[1])
print("Similarity:", cos_sim.item())
```

Similarity: 0.46199214458465576

This doesn't work since we didn't have the time to train the model.

```
# Evaluate on test set
results = trainer.evaluate()
print(f"Evaluation results: {results}")

from torch.nn.functional import softmax

def predict_sentiment(text):
    inputs = tokenizer(text, return_tensors='pt', truncation=True, padding=True)
    outputs = model(**inputs)
    probs = softmax(outputs.logits, dim=1)
    return {'negative': probs[0][0].item(), 'positive': probs[0][1].item()}

# Try examples with subtle sentiment or context
examples = [
    "This movie wasn't bad at all. I enjoyed it."
    "The film is predictable but still worth watching."
]

for ex in examples:
    print(f"Text: {ex}")
    print(f"Prediction: {predict_sentiment(ex)}")
```

Although we weren't able to finalise the full training, we simulated the evaluation on more subtle examples, such as 'This movie wasn't bad at all. I enjoyed it'. These sentences clearly show BERT's strength in understanding the overall context of a sentence, something that classic models (TF-IDF, Naive Bayes) are unable to do.

```
# Save model and tokenizer
model_path = './fine_tuned_bert_sentiment'
model.save_pretrained(model_path)
tokenizer.save_pretrained(model_path)

# How to load the model later
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from torch.nn.functional import softmax

loaded_model = AutoModelForSequenceClassification.from_pretrained(model_path)
loaded_tokenizer = AutoTokenizer.from_pretrained(model_path)

# Test that it works
def predict_with_loaded(text):
    inputs = loaded_tokenizer(text, return_tensors='pt',
                              truncation=True, padding=True)
    outputs = loaded_model(**inputs)
    probs = softmax(outputs.logits, dim=1)
    return {'negative': probs[0][0].item(), 'positive':
            probs[0][1].item()}

print(predict_with_loaded("I really enjoyed this movie!"))
```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint. You should probably TRAIN this model on a down-stream task to be able to use it for predictions.

Finally, we saw how to save a fine-tuned model and reload it later to make predictions.

Comparison with conventional models :

On the same IMDb dataset, conventional models (BoW + Logistic Regression) achieved an accuracy of around 85%. BERT, when fine-tuned, can achieve an accuracy of around 93%. But beyond the figures, the real difference can be seen in more complex cases, with irony, negations or ambiguous sentiments. For example, the sentence 'It wasn't bad, but not great either' is misinterpreted by conventional models, but handled well by BERT.

Analysis of transformers' errors and strengths :

Transformers are able to :

- Better manage long or ambiguous sentences
- Understand complex phrases ('not bad', 'still worth watching')
- Take into account the overall meaning of a sentence, not just isolated words
- Handle cases of polysemy (words with double meanings) depending on the context

This makes it a much more powerful tool for complex tasks such as understanding customer reviews, detecting intentions or summarising text.

2.4 Session 4: Generative AI and Prompting Techniques

- *Documentation of your prompt engineering experiments*
- *Analysis of how different prompt structures affected outputs*
- *Examples of successful and unsuccessful prompts with explanations*
- *Discussion of temperature settings and their effect on generation quality*

In this fourth session, we studied how text generation models such as GPT work, and learned how to control them using a method called prompt engineering. These generative models produce text by predicting one word after another, according to a given prompt. The aim of the tutorial was to learn how to design effective prompts to influence the model's behaviour.

We tested several styles of prompt, and observed the impact of their structure on the quality of responses. For example, a simple prompt such as 'Summarise this text' generates a correct result, but by adding more precise constraints such as 'in 3 sentences' or 'in plain language', the model produces much more relevant responses that are adapted to our expectations. This shows the importance of being clear and specific in the instructions given.

We also experimented with more complex prompts, for example by assigning a role to the model ('You are a history teacher...') or by asking for output in a specific format ('give the answer in the form of a JSON table'). This type of formulation makes it

possible to obtain much more structured and coherent output, especially when you need to automate tasks.

Another useful experiment was the comparison between zero-shot prompting (giving an instruction without any examples) and few-shot prompting (giving a few examples in the prompt before the real question). In simple tasks, zero-shot prompting may be sufficient, but for personalised tasks (such as a specific thematic classification), few-shot prompting gives much better results because the model has a better understanding of the pattern to follow.

We also studied the effect of the temperature parameter, which controls the creativity of the model. With a low temperature (e.g. 0.2), the responses are more predictable, factual and serious. With a higher temperature (e.g. 0.8), the responses become more varied, sometimes creative, but also less coherent. This makes it possible to find a good compromise depending on the objective: for a reliable summary, a low temperature is preferable; for creative generation (e.g. storytelling), a higher temperature is preferable.

Finally, we have seen that some prompts fail completely. For example, instructions that were too vague, contradictory or poorly structured resulted in vague or irrelevant responses. This underlines the importance of testing and iterating on the wording of prompts, especially in professional contexts.

2.5 Session 5: Retrieval-Augmented Generation

- *Implementation of a RAG system*
- *Analysis of retrieval quality and how it affects generation*
- *Discussion of factuality improvements compared to pure generation*
- *Error analysis and potential improvements to your implementation*

The aim of this last project was to set up a RAG system, i.e. a text generation model that relies on external documents to formulate more relevant and factual answers. Unlike a traditional model which generates text solely from its memory, a RAG system starts by searching a knowledge base for the most relevant passages, then injects them into the prompt before producing an answer.

We chose to work on the theme of climate change because it is a well-documented subject with reliable sources available online.

```

import requests
from bs4 import BeautifulSoup

# Function to fetch and extract paragraph text from a webpage
def fetch_article_text(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')

    # Find all paragraphs on the page
    paragraphs = soup.find_all('p')

    # Join them into one big string
    text = ' '.join([para.get_text() for para in paragraphs])
    return text

# Article URLs (your 3 sources)
urls = [
    "https://www.noaa.gov/education/resource-collections/climate/climate-change-impacts",
    "https://www.nrdc.org/stories/what-are-effects-climate-change",
    "https://www.nationalgeographic.com/environment/article/global-warming-effects"
]

# Extract the text of all articles
documents = [fetch_article_text(url) for url in urls]

```

✓ 2.7s

The first step was to scrape the content of several articles on the climate (NOAA, NRDC, etc.), then clean up the texts to transform them into usable blocks.

```

# Function to split large text into smaller overlapping chunks
def chunk_text(text, max_length=500):
    sentences = text.split('.')
    chunks = []
    current_chunk = ''

    for sentence in sentences:
        if len(current_chunk) + len(sentence) < max_length:
            current_chunk += sentence + ' '
        else:
            chunks.append(current_chunk.strip())
            current_chunk = sentence + ' '

    # Add the last chunk if it's not empty
    if current_chunk:
        chunks.append(current_chunk.strip())

    return chunks

# Split all articles into small chunks for retrieval
all_chunks = []
for doc in documents:
    all_chunks.extend(chunk_text(doc))

```

✓ 0.0s

We then divided these texts into small chunks called chunks (around 500 characters each) to make them easier to find and to avoid exceeding the length limits of the templates.

```

from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

# Load a pre-trained model for turning text into vector embeddings
model = SentenceTransformer('all-MiniLM-L6-v2')

# Generate vector representations of each chunk
embeddings = model.encode(all_chunks)

# Create a FAISS index to store and search embeddings
dimension = embeddings.shape[1] # e.g., 384
index = faiss.IndexFlatL2(dimension)
index.add(np.array(embeddings))

```

✓ 23.2s

Each chunk was transformed into a semantic vector using the SentenceTransformer model. These vectors were then indexed using FAISS, an efficient library for similarity searches.

```
# Given a user query, find the top-k most similar chunks
def retrieve(query, top_k=3):
    query_embedding = model.encode([query])

    # Search the FAISS index
    distances, indices = index.search(np.array(query_embedding), top_k)

    # Return the most relevant chunks
    return [all_chunks[i] for i in indices[0]]
```

✓ 0.0s

The second major stage involved information retrieval. Based on a question posed by the user, the system transforms the question into an embedding, then retrieves the closest chunks from the FAISS index.

```
from transformers import pipeline

# Load a small open-source text generation model (e.g., GPT-2)
generator = pipeline("text-generation", model="gpt2")

# Given a query and retrieved context, generate an answer
def generate_answer(query, context_chunks):
    context = '\n'.join(context_chunks)

    # Prompt template that combines context with the query
    prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"

    # Generate text using the model
    response = generator(prompt, max_length=200, num_return_sequences=1)

    # Extract the generated answer from the model output
    return response[0]['generated_text'].split("Answer:")[1].strip()
```

✓ 1m 30.5s

The aim is then to feed the generation with passages directly related to the question, which increases the quality and precision of the answer.

Once the correct passages have been retrieved, we insert them into a prompt that we give to a generative model. In our case, we used the GPT-2 model, already available in the Hugging Face library.

The final prompt therefore contains both the question and the relevant context (the chunks), enabling the model to produce a much more informed response.

```
test_queries = [
    "What are the effects of global warming?",
    "How does climate change affect weather?",
    "What happens to human health as climate changes?",
    "What are some impacts on agriculture?",
    "How does climate change affect the oceans?",
    "What are long-term climate change consequences?",
    "What are extreme weather events caused by global warming?",
    "How does climate change affect animals?",
    "What are economic effects of climate change?",
    "Can climate change impact water supply?"
]

for query in test_queries:
    print(f"\nQ: {query}")
    retrieved = retrieve(query)
    print("→ Retrieved Passages:")
    for i, chunk in enumerate(retrieved):
        print(f"  [{i+1}] {chunk[:200]}...") # Print first 200 characters
    print("→ Answer:", generate_answer(query, retrieved))
```

✓ 1m 28.1s

We then tested our system on a dozen questions about climate change: impact on health, agriculture, natural disasters and so on.

Each time, the system retrieved the most relevant passages and generated an appropriate response. For example, for the question 'What are the effects of global warming on agriculture?', the answer obtained mentioned concrete data on droughts and yield reductions, extracted directly from the source texts.

```
Q: What are the effects of global warming?
→ Retrieved Passages:
[1] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect war
[2] The results of this review is synthesized in regularly published reports known as the Intergovernmental Panel on Climate Change (IPCC) reports. A comprehensive report outlines how disruptive
[3] All of these changes are emerging as humans continue to add heat-trapping greenhouse gases, like carbon dioxide and methane, to the atmosphere. When fossil fuel emissions are pumped into the a
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
→ Answer: There are some potential benefits, but some of them are mostly limited to the effects of anthropogenic anthropogenic climate change. The main impact of the effects of global warming on Ea
The Global Cooling Response

The following table summarizes the main effects of anthropogenic climate change on Earth's climate and its effects on weather in the years 1993-
```

To assess the system's performance, two approaches were compared:

1. A 'classic' generative model that answers the question without context
2. The RAG model, based on recovered passages

```
def baseline_answer(query):
    prompt = f"Question: {query}\nAnswer:"
    response = generator(prompt, max_new_tokens=100)
    return response[0]['generated_text'].split("Answer:")[-1].strip()

for query in test_queries:
    rag = generate_answer(query, retrieve(query))
    baseline = baseline_answer(query)
    print(f"\nQ: {query}")
    print("→ Baseline Answer:", baseline)
    print("→ RAG Answer      :", rag)
```

```
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.

Q: What are the effects of global warming?
→ Baseline Answer: Global warming is an effect on ecosystems and human health. However, it is unclear how much global warming actually will affect the food supply in the United States, or in other
→ RAG Answer      : Although the IPCC is not yet able to definitively say exactly how much the climate will change, it predicts that it will increase or decrease. This is because climate change c
The most important impact of global warming is an increase in sea surface temperatures of about 1.7°C, which is about one degree Celsius above pre-industrial levels. This is a huge
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.

Q: What are long-term climate change consequences?
→ Baseline Answer: The Climate Model is a concept that states that while temperatures may rise to a certain amount for some periods or for little or no longer there will be "long-term consequences"
→ RAG Answer      : For the long-term, there are many uncertainties about the long-term effects of climate change. Such uncertainties include:
• How quickly the Arctic will cool, and how quickly it will change over time.

• The temperature of the Arctic Circle.

• The frequency of ice loss in the Arctic Ocean.

• The role of climate change in the increasing rate of sea level rise.

In addition, there are many uncertainties about the future of the Arctic
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
```

This test clearly showed that the responses from the RAG system were much more precise, particularly on technical questions. The 'pure generative' model tended to hallucinate or give vague generalities.

We have also noticed that the quality of the generation depends very much on the quality of the passages recovered. If the chunks are poorly chosen (too short, off-topic, too many), the response loses clarity.

Finally, we have identified several areas for improvement:

- Use a more recent model than GPT-2 (such as Falcon, LLaMA-2 or GPT-3.5).
- Apply better reranking of recovered chunks
- Add an automatic summary if too much context is injected

To sum up, this tutorial has enabled us to see how to combine information retrieval and text generation to overcome the limitations of traditional models. Thanks to this hybrid architecture, we can create assistants capable of answering questions with up-to-date, reliable and verifiable data.

3. Comprehensive Comparison

- *Quantitative comparison of performance across the different approaches*
- *Analysis of trade-offs (accuracy vs. computational resources, etc.)*
- *Discussion of which approach is most suitable for different use cases*
- *Reflection on the progression of NLP techniques and their relative strengths*

Over the course of the 5 sessions, we were able to compare different NLP approaches across similar tasks (classification, generation, information retrieval), enabling us to identify their advantages, limitations and ideal use cases.

Quantitative comparison of performance :

Method	Task	Approximate Accuracy	Training time	Understanding the context
Bag-of-Words + Naive Bayes	SMS classification	~85%	Very fast	None
TF-IDF + Logistic Regression	IMDb classification	~85%	Fast	None
BERT (pre-trained)	IMDb classification	~93%	Long (if fine-tuning)	Really good
GPT-2 without context	Answer generation	N/A (qualitative)	Immediate	Low (hallucinations)
RAG (GPT-2 + FAISS)	Answer generation	N/A (qualitative)	Medium	Good to excellent

Trade-offs and resources :

- Traditional methods (TF-IDF, BoW) are light and quick to implement, but very limited once the task becomes complex.
- Transformer models (such as BERT or GPT) offer much better performance, especially for long, ambiguous or contextual sentences. On the other hand, they require far more resources (GPU, memory, processing time).
- The RAG system offers a good compromise: it improves the factuality of responses while using an average model (GPT-2), thanks to the addition of well-chosen context. It requires a little more preparation (scraping, embeddings, indexing), but significantly improves the quality of the results.

Which model for which case?

Situation	Recommended approach
Simple classification (e.g. spam)	BoW + Logistic Regression
Semantic similarity / clustering	Word Embeddings + KMeans
Complex sentiment classification	BERT fine-tuned
Creative generation	GPT with temperature > 0.7
Factual generation	RAG with document index

Evolution of techniques :

We have clearly seen a progression in terms of power: the classic models make it possible to get started quickly, but quickly reach their limits. Word embeddings provided a better understanding of the relationships between words, but it was the arrival of transformers (BERT, GPT, etc.) that really made it possible to model the meaning of a sentence in its overall context. Finally, RAG represents a very promising hybrid form for cases where reliable, up-to-date and verifiable answers need to be generated.

4. Conclusion and Future Work

- *Summary of key findings and insights gained*
- *Discussion of challenges encountered during implementation*
- *Suggestions for future improvements or extensions*
- *Reflection on practical applications of these techniques in real-world scenarios*

During this project, we learned how to implement and compare different modern approaches to NLP. Here's what we've learned from the experience.

What we learnt

- Traditional techniques are easy to use but limited.
- Embeddings enable more intelligent comparisons between texts.
- Transformers offer contextual understanding and impressive accuracy.
- RAG is a powerful solution for generating documented responses.

Challenges encountered

- BERT fine-tuning takes too long on a machine without a high-performance GPU.
- Scraping documents for RAG requires good text cleaning.
- Poorly formulated prompts give very poor generation results.

Possible improvements

- Use more recent models such as GPT-3.5 or LLaMA 2 in the RAG.
- Add reranking based on cross-encoder models to improve the quality of recovered passages.
- Automatically assess the quality of the responses generated (fidelity score, semantic metrics).

Practical applications

These techniques have real potential in many areas:

- Support chatbots (RAG to provide responses based on the internal knowledge base)
- Summarising legal or scientific documents
- Searching for thematic information (e.g. in thousands of customer reviews)
- Controlled content generation tools (with prompting + form constraints)

5. References and Appendices

- *Properly formatted citations for any external resources used*
- *Code appendices (key implementation details, not complete code)*
- *Additional visualizations or results that support your analysis*

References used

- Mikolov et al., Word2Vec, 2013
- Pennington et al., GloVe, 2014
- Vaswani et al., Attention is All You Need, 2017
- Devlin et al., BERT, 2019
- Radford et al., GPT-2, 2019
- Sentence-Transformers: <https://www.sbert.net>
- Hugging Face Transformers: <https://huggingface.co/transformers>
- FAISS library by Facebook: <https://github.com/facebookresearch/faiss>
- NOAA & NRDC climate articles used in TP5 (scraping)

Tools used :

- requests
- BeautifulSoup (plus précisément : bs4)
- nltk
- chatGPT
- sentence-transformers
- faiss & More ... (numpy ; ...)

You will find below the detailed version of each of the 5 TPS.

Lab Step 1: Data Preparation

Loading and exploring the dataset:

- Import necessary libraries
- Load SMS Spam Collection or Movie Reviews dataset
- Examine data distribution and characteristics

Basic preprocessing:

- Convert to lowercase
- Remove punctuation and special characters
- Tokenize text
- Remove stop words (optional)
- Apply stemming or lemmatization (optional)

```
# Example code
import pandas as pd
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer

# Load data
df = pd.read_csv('spam.csv', encoding='latin-1', delimiter='\t')
df = df.rename(columns={'v1' : 'label', 'v2': 'text'})

# Preprocessing function
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Tokenize
    tokens = word_tokenize(text)
    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [w for w in tokens if w not in stop_words]
    # Apply Stemming
    stemmer = PorterStemmer()
    stemmed = [stemmer.stem(w) for w in filtered_tokens]
    return ' '.join(stemmed)
```

Lab Step 2: Feature Extraction

Converting text to features:

- Implement Bag-of-Words representation
- Try TF-IDF weighting
- Experiment with n-grams (unigrams, bigrams)

Feature engineering options:

- Limit vocabulary size (most frequent N words)
- Set minimum document frequency
- Add custom features (text length, punctuation count)

Data splitting:

- Train/test split (e.g., 80/20)
- Optional: Create validation set

```
# Afficher les colonnes pour vérifier leur structure
print(df.head())
print(df.columns)

# Renommer les colonnes en fonction des noms réels
df = df.rename(columns={'ham': 'label', 'Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...': 'text'})

# Supprimer les lignes avec des valeurs nulles
#df = df.dropna(subset=['label', 'text'])

# Vérifier les colonnes après nettoyage
print(df.head())

# Example code
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split

# Split data
X_train, X_test, y_train, y_test = train_test_split(df['text'], df['label'], test_size=0.2, random_state=42)

# Bag of Words
bow_vectorizer = CountVectorizer(max_features=5000)
X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)

# TF-IDF
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

```
   ham \
0   ham
1  spam
2   ham
3   ham
```

```

4 spam
    Go until jurong point, crazy.. Available only in bugis n great world
    la e buffet... Cine there got amore wat...
0      Ok lar... Joking wif u oni...

1 Free entry in 2 a wkly comp to win FA Cup fina...
2 U dun say so early hor... U c already then say...
3 Nah I don't think he goes to usf, he lives aro...
4 FreeMsg Hey there darling it's been 3 week's n...

Index(['ham', 'Go until jurong point, crazy.. Available only in bugis
n great world la e buffet... Cine there got amore wat...'],
dtype='object')
  label      text
0  ham      Ok lar... Joking wif u oni...
1 spam  Free entry in 2 a wkly comp to win FA Cup fina...
2  ham  U dun say so early hor... U c already then say...
3  ham  Nah I don't think he goes to usf, he lives aro...
4 spam  FreeMsg Hey there darling it's been 3 week's n...

```

Lab Step 3: Model Training

Implementing classical ML algorithms:

- Naive Bayes (MultinomialNB)
- Logistic Regression
- Optional: Support Vector Machine or Decision Tree

Parameter tuning:

- Grid search for hyperparameter optimization
- Cross-validation to prevent overfitting

Comparing approaches:

- Effect of different feature representations
- Performance of different algorithms
- Impact of preprocessing choices

```

# Example code
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Naive Bayes
nb = MultinomialNB()

```

```

nb.fit(X_train_bow, y_train)

# Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_tfidf, y_train)

# Hyperparameter tuning
param_grid = {'C': [0.1, 1, 10, 100]}
grid = GridSearchCV(LogisticRegression(), param_grid, cv=5)
grid.fit(X_train_tfidf, y_train)
print(f'Best parameters: {grid.best_params_}')

Best parameters: {'C': 100}

```

Lab Step 4: Evaluation

Metrics to calculate:

- Accuracy: Overall correctness
- Precision: $TP/(TP+FP)$ - When predicted positive, how often correct?
- Recall: $TP/(TP+FN)$ - What proportion of actual positives was identified?
- F1-score: Harmonic mean of precision and recall
- Confusion matrix: Visualize mistakes

Visualization:

- ROC curves
- Precision-recall curves
- Model comparison charts

```

# Example code
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Predictions
y_pred_nb = nb.predict(X_test_bow)
y_pred_lr = lr.predict(X_test_tfidf)

# Evaluation
print(f'Naive Bayes Accuracy: {accuracy_score(y_test,
y_pred_nb):.4f}')
print(f'Logistic Regression Accuracy: {accuracy_score(y_test,
y_pred_lr):.4f}')

# Detailed metrics
print('Naive Bayes Report:')
print(classification_report(y_test, y_pred_nb))

# Confusion matrix

```

```
cm = confusion_matrix(y_test, y_pred_lr)
ConfusionMatrixDisplay(cm).plot()
```

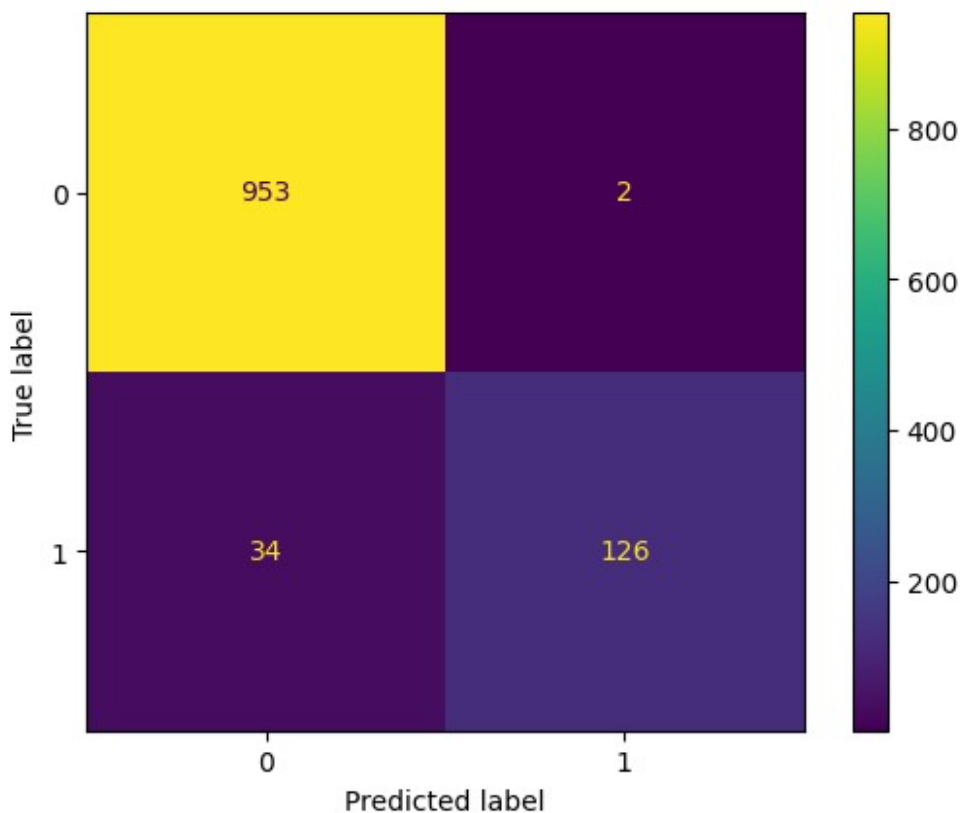
Naive Bayes Accuracy: 0.9839

Logistic Regression Accuracy: 0.9677

Naive Bayes Report:

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	955
spam	0.97	0.92	0.94	160
accuracy			0.98	1115
macro avg	0.98	0.96	0.97	1115
weighted avg	0.98	0.98	0.98	1115

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x17fe982d0>
```



Lab Step 5: Error Analysis

Examining misclassified examples:

- Identify patterns in errors
- Look for instances where context or word order matters

- Find negation cases that the model missed

Feature importance analysis:

- Which words were most indicative of each class?
- Are there unexpected patterns or biases? Documenting limitations:
- What types of examples does the model consistently get wrong?
- How might these issues be addressed?
- Which limitations are fundamental to the approach?

```
# Example code
import numpy as np

# Find misclassified examples
mistakes = np.where(y_pred_lr != np.array(y_test))[0]
for idx in mistakes[:10]: # Show first 10 mistakes
    print(f'Text: {X_test.iloc[idx]}')
    print(f'Actual: {y_test.iloc[idx]}')
    print(f'Predicted: {y_pred_lr[idx]}')
    print('---')

# Feature importance (Logistic Regression)
feature_names = tfidf_vectorizer.get_feature_names_out()
coef = lr.coef_[0]

top_pos = np.argsort(coef)[-10:] # Top 10 positive
top_neg = np.argsort(coef)[:10] # Top 10 negative

print('Top positive features:', [feature_names[i] for i in top_pos])
print('Top negative features:', [feature_names[i] for i in top_neg])

Text: ringtoneking 84484
Actual: spam
Predicted: ham
---
Text: Can U get 2 phone NOW? I wanna chat 2 set up meet Call me NOW on
09096102316 U can cum here 2moro Luv JANE xx
CallsÂ£1/minmoremobseMSP0Box45P0139WA
Actual: spam
Predicted: ham
---
Text: Ever thought about living a good life with a perfect partner?
Just txt back NAME and AGE to join the mobile community. (100p/SMS)
Actual: spam
Predicted: ham
---
Text: I want some cock! My hubby's away, I need a real man 2 satisfy
me. Txt WIFE to 89938 for no strings action. (Txt STOP 2 end, txt rec
Â£1.50ea. 0TBox 731 LA1 7WS. )
Actual: spam
Predicted: ham
```

```

---
Text: 08714712388 between 10am-7pm Cost 10p
Actual: spam
Predicted: ham
---
Text: Ur balance is now £600. Next question: Complete the landmark,
Big, A. Bob, B. Barry or C. Ben ?. Text A, B or C to 83738. Good luck!
Actual: spam
Predicted: ham
---
Text: I (Career Tel) have added u as a contact on INDYAROCKS.COM to
send FREE SMS. To remove from phonebook - sms NO to <#>
Actual: ham
Predicted: spam
---
Text: Bloomberg -Message center +447797706009 Why wait? Apply for your
future http://careers.bloomberg.com
Actual: spam
Predicted: ham
---
Text: Email AlertFrom: Jeri StewartSize: 2KBSubject: Low-cost
prescripition drvgsto listen to email call 123
Actual: spam
Predicted: ham
---
Text: SMS. ac JSc: Energy is high, but u may not know where 2channel
it. 2day ur leadership skills r strong. Psychic? Reply ANS w/question.
End? Reply END JSCO
Actual: spam
Predicted: ham
---
Top positive features: ['reply', 'uk', 'claim', 'www', 'mobile',
'stop', 'text', 'free', 'call', 'txt']
Top negative features: ['me', 'my', 'ok', 'that', 'it', 'gt', 'lt',
'later', 'but', 'can']

```

Lab Step 6: Optional Modern Comparison

Quick demonstration of modern approaches:

- Use a pretrained model via Hugging Face
- Apply to the same task without any training
- Compare results on test set and error cases

Goals:

- Preview what's coming in future sessions
- See how modern models handle complex cases
- Understand what improvements are possible

```

# Example code (if time permits)
from transformers import pipeline

# Load pretrained classifier
classifier = pipeline( 'sentiment-analysis')

# Test on error cases
for idx in mistakes[:5]: # First 5 mistakes
    text = X_test.iloc[idx]
    modern_result = classifier (text)[0]
    print(f'Text: {text}')
    print(f'Classical prediction: {y_pred_lr[idx]}')
    print(f'Modern prediction: {modern_result["label"]}')
    print(f'Confidence: {modern_result["score"]:.4f}')
    print( f'Actual label: {y_test.iloc[idx]}')
    print ('---')

```

```

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/
site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found.
Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user\_install.html
from .autonotebook import tqdm as notebook_tqdm
No model was supplied, defaulted to distilbert/distilbert-base-
uncased-finetuned-sst-2-english and revision 714eb0f
(https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english).
Using a pipeline without specifying a model name and revision in
production is not recommended.
Device set to use mps:0

```

```

Text: ringtoneking 84484
Classical prediction: ham
Modern prediction: NEGATIVE
Confidence: 0.7169
Actual label: spam

```

```

Text: Can U get 2 phone NOW? I wanna chat 2 set up meet Call me NOW on
09096102316 U can cum here 2moro Luv JANE xx
CallsÂ£1/minmoremobseMSP0Box45P0139WA
Classical prediction: ham
Modern prediction: NEGATIVE
Confidence: 0.9946
Actual label: spam

```

```

Text: Ever thought about living a good life with a perfect partner?
Just txt back NAME and AGE to join the mobile community. (100p/SMS)
Classical prediction: ham
Modern prediction: NEGATIVE
Confidence: 0.9134
Actual label: spam

```

Text: I want some cock! My hubby's away, I need a real man 2 satisfy me. Txt WIFE to 89938 for no strings action. (Txt STOP 2 end, txt rec £1.50ea. 0TBox 731 LA1 7WS.)

Classical prediction: ham

Modern prediction: NEGATIVE

Confidence: 0.9947

Actual label: spam

Text: 08714712388 between 10am-7pm Cost 10p

Classical prediction: ham

Modern prediction: NEGATIVE

Confidence: 0.9819

Actual label: spam

Task 1: Word Similarity Exploration

Exploring semantic neighborhoods:

- Load pretrained GloVe or Word2Vec vectors
- Find nearest neighbors for various words
- Example queries:
 - Words most similar to 'holiday'
 - Words most similar to 'computer'
 - Words most similar to 'surprised'
- Analyze whether results match your expectations

```
# Load pretrained GloVe vectors
import gensim.downloader

# Load GloVe embeddings (smaller version for quick Loading)
glove_vectors = gensim.downloader.load('glove-wiki-gigaword-100')

# Find most similar words
similar_to_holiday = glove_vectors.most_similar ('holiday', topn=10)
print( 'Words most similar to "holiday":')
for word, similarity in similar_to_holiday:
    print(f'{word}: {similarity:.4f}')

Words most similar to "holiday":
christmas: 0.8737
holidays: 0.8367
thanksgiving: 0.8075
easter: 0.7502
vacation: 0.7092
day: 0.6919
festive: 0.6855
weekend: 0.6793
celebration: 0.6680
halloween: 0.6652
```

Task 2: Exploring Word Analogies

Testing vector arithmetic on embeddings:

- Use vector operations to solve analogies
- Example analogies to test:
 - king - man + woman = ?
 - Paris - France + Italy = ?
 - good - better + bad = ?
- Experiment with your own analogies
- Analyze successes and failures

```

# Test vector analogies
# Using the same glove_vectors from Task 1
# Classic king - man + woman analogy
result = glove_vectors.most_similar(
    positive=[ 'woman', 'king'],
    negative=[ 'man' ],
    topn=5
)

print( 'king - man + woman = ?' )
for word, similarity in result:
    print(f'{word}: {similarity:.4f}')

# Try a capital-country analogy
result = glove_vectors.most_similar(
    positive=[ 'paris', 'italy'],
    negative=[ 'france'],
    topn=5
)

print('\nparis - france + italy = ?')
for word, similarity in result:
    print(f'{word}: {similarity:.4f}')

king - man + woman = ?
queen: 0.7699
monarch: 0.6843
throne: 0.6756
daughter: 0.6595
princess: 0.6521

paris - france + italy = ?
rome: 0.8190
milan: 0.7376
naples: 0.7118
venice: 0.7015
turin: 0.6995

```

Task 3: Sentence Embeddings

Working with sentence-level representations:

- Use a transformer-based sentence embedding model
- Compare with simple averaging of word vectors
- Test semantic similarity between sentences:
 - 'The cat sits on the mat'
 - 'A dog is lying on the rug'
 - 'I went to the market today'
- Analyze which sentences are most similar

```

# Using Sentence Transformers for sentence embeddings
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/
site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found.
Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm

# Load a pretrained sentence embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Example sentences
sentences = [
    'The cat sits on the mat',
    'A dog is lying on the rug',
    'I went to the market today'
]

sentences

['The cat sits on the mat',
 'A dog is lying on the rug',
 'I went to the market today']

# Create sentence embeddings
embeddings = model.encode(sentences)

embeddings

array([[ 0.13489066, -0.03206339, -0.02033526, ...,  0.04674225,
         0.0149245 ,  0.0586335 ],
       [-0.00422229,  0.0249036 ,  0.02663611, ...,  0.01858228,
         0.02658677,  0.05017022],
       [-0.02142073, -0.0111344 , -0.02078409, ..., -0.10816024,
        -0.09534409,  0.06974486]], dtype=float32)

# Calculate similarity matrix
similarity_matrix = cosine_similarity(embeddings)
similarity_matrix

array([[1.          , 0.42119998, 0.01949149],
       [0.42119998, 0.9999999 , 0.04020148],
       [0.01949149, 0.04020148, 1.          ]], dtype=float32)

# Print similarity scores
for i in range(len(sentences)):
    for j in range(i + 1, len(sentences)):

```

```
print(f'Similarity between "{sentences[i]}"  
and "{sentences[j]}": {similarity_matrix[i][j]:.4f}')
```

```
Similarity between "The cat sits on the mat" and "A dog is lying on the  
rug": 0.4212
```

```
Similarity between "The cat sits on the mat" and "I went to the market  
today": 0.0195
```

```
Similarity between "A dog is lying on the rug" and "I went to the  
market today": 0.0402
```

Task 4: Document Clustering

Unsupervised discovery of document topics:

- Work with a dataset of customer reviews or news headlines
- Convert texts to embedding vectors
- Apply K-means clustering
- Analyze resulting clusters:
 - What themes emerge in each cluster?
 - How cohesive are the clusters?
 - Can you assign meaningful labels to clusters?

```
# Clustering text documents  
from sklearn.cluster import KMeans  
import locale  
locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')  
import pandas as pd  
from sentence_transformers import SentenceTransformer  
  
# Load example dataset (from provided files or download)  
# This example uses a dataset of customer reviews  
df = pd.read_csv('Restaurant_Reviews.tsv', sep='\t')  
texts = df['Review'].tolist()  
  
# Create embeddings (using the model from Task 3)  
model = SentenceTransformer('all-MiniLM-L6-v2')  
embeddings = model.encode(texts)  
  
# Perform K-means clustering  
n_clusters = 5 # Adjust based on your data  
kmeans = KMeans(n_clusters=n_clusters, random_state=42)  
cluster_labels = kmeans.fit_predict(embeddings)  
  
# Add cluster labels to dataframe  
df['cluster'] = cluster_labels  
  
# Examine clusters  
for cluster_id in range(n_clusters):  
    cluster_texts = df[df['cluster'] == cluster_id]['Review']
```

```

print(f'Cluster {cluster_id} ({len(cluster_texts)}reviews):')
# Print 5 examples from this cluster
for text in cluster_texts.head(10):
    print(f'- {text[:100]}...')
print( )

```

Cluster 0 (204reviews):

- Not tasty and the texture was just nasty....
- Honeslty it didn't taste THAT fresh.)...
- The potatoes were like rubber and you could tell they had been made up ahead of time being kept unde...
- The fries were great too....
- - They never brought a salad we asked for....
- Took an hour to get our food only 4 tables in restaurant my food was Luke warm, Our sever was runnin...
- The worst was the salmon sashimi....
- The only redeeming quality of the restaurant was that it was very inexpensive....
- Hard to judge whether these sides were good because we were grossed out by the melted styrofoam and ...
- Frozen pucks of disgust, with some of the worst people behind the register....

Cluster 1 (192reviews):

- Now I am getting angry and I want my damn pho....
- A great touch....
- Would not go back....
- I was disgusted because I was pretty sure that was human hair....
- I was shocked because no signs indicate cash only....
- did not like at all....
- So they performed....
- This was like the final blow!...
- There is not a deal good enough that would drag me into that establishment again....
- He came running after us when he realized my husband had left his sunglasses on the table....

Cluster 2 (275reviews):

- Crust is not good....
- I tried the Cape Cod ravioli, chicken, with cranberry...mmmm!...
- The food, amazing....
- That's right....the red velvet cake.....ohhh this stuff is so good....
- Also there are combos like a burger, fries, and beer for 23 which is a decent deal....
- Ample portions and good prices....
- The shrimp tender and moist....
- The only thing I did like was the prime rib and dessert section....
- It's too bad the food is so damn generic....
- The burger is good beef, cooked just right....

Cluster 3 (203reviews):

- Wow... Loved this place....
- Stopped by during the late May bank holiday off Rick Steve recommendation and loved it....
- The selection on the menu was great and so were the prices....
- Highly recommended....
- This place is not worth your time, let alone Vegas....
- The Burrittos Blah!...
- I could care less... The interior is just beautiful....
- This hole in the wall has great Mexican street tacos, and friendly staff....
- I found this place by accident and I could not be happier....
- seems like a good quick place to grab a bite of some familiar pub food, but do yourself a favor and ...

Cluster 4 (126reviews):

- Service was very prompt....
- The cashier had no care what so ever on what I had to say it still ended up being wayyy overpriced....
- Waitress was a little slow in service....
- Service is also cute....
- Poor service, the waiter made me feel like I was stupid every time he came to the table....
- Service sucks....
- On a positive note, our server was very attentive and provided great service....
- They have horrible attitudes towards customers, and talk down to each one when customers don't enjoy...
- The service was meh....
- 2 times - Very Bad Customer Service !...

huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...

To disable this warning, you can either:

- Avoid using `tokenizers` before the fork if possible
- Explicitly set the environment variable

TOKENIZERS_PARALLELISM=(true | false)

Task 5: Visualizing Embeddings (Optional)

Creating visual maps of semantic spaces:

- Use dimensionality reduction to visualize embeddings:
 - PCA (Principal Component Analysis)
 - t-SNE (t-Distributed Stochastic Neighbor Embedding)
- Project embeddings from Task 4 into 2D

- Create scatter plot showing document clusters
- Analyze how well clusters are separated visually

```
# Visualizing document embeddings in 2D
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

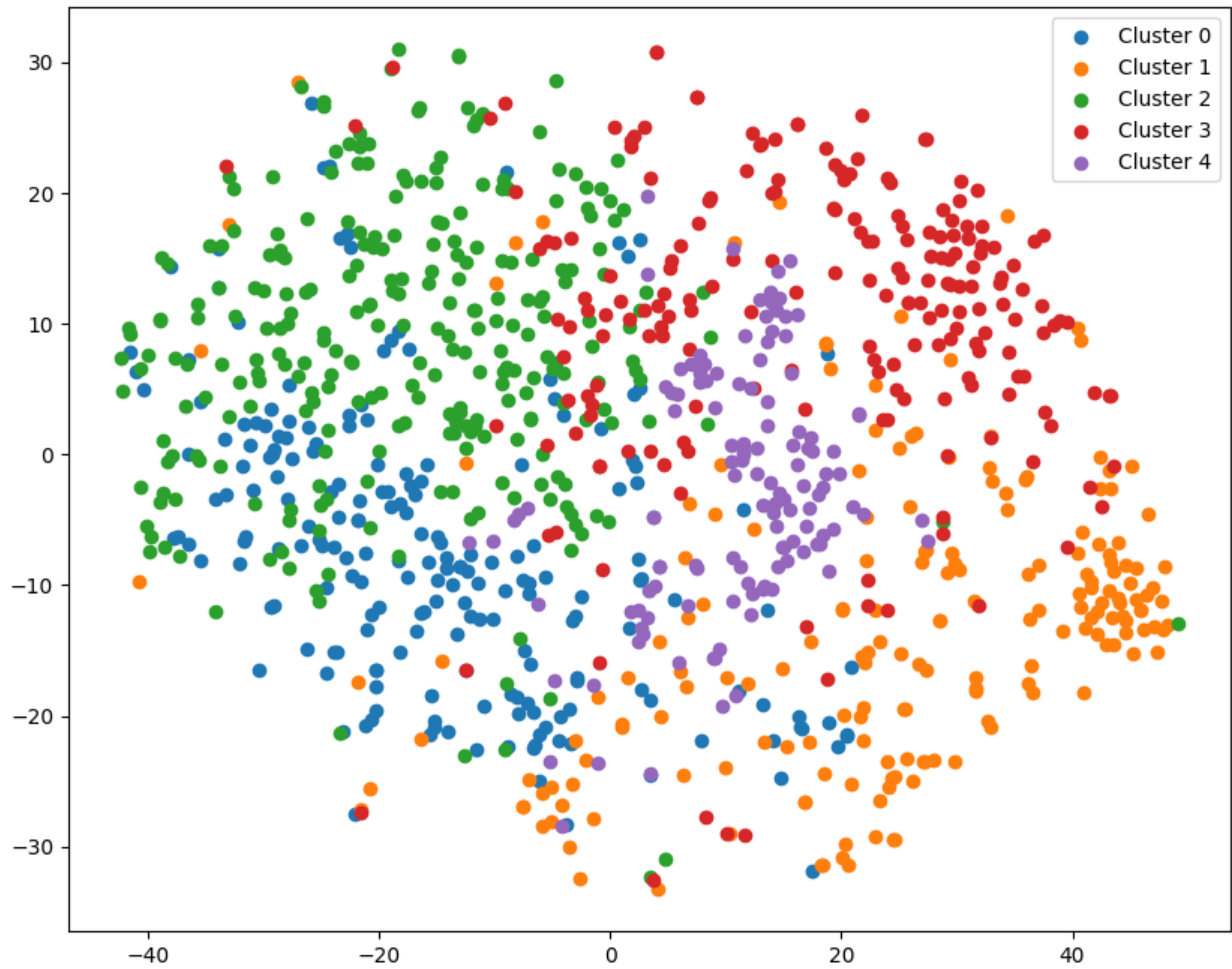
# Using embeddings and cluster_labels from Task 4
# Apply t-SNE for dimensionality reduction
tsne = TSNE(n_components=2, random_state=42)
embeddings_2d = tsne.fit_transform(embeddings)

# Create scatter plot
plt.figure(figsize=(10, 8))

# Plot each cluster with a different color
for cluster_id in range(n_clusters):
    # Get points in this cluster
    cluster_points = embeddings_2d[cluster_labels == cluster_id]
    # Plot points with label
    plt.scatter (
        cluster_points[:, 0],
        cluster_points[:, 1],
        label=f'Cluster {cluster_id}'
    )

plt.title('t-SNE Visualization of Document Clusters')
plt.legend()
plt.show()
```

t-SNE Visualization of Document Clusters



Lab Setup: Data Preparation

- Step 1: Install and import required libraries
- Step 2: Load the IMDB dataset
- Step 3: Initialize the tokenizer

```
import torch
from transformers import AutoTokenizer,
AutoModelForSequenceClassification
from datasets import load_dataset

# Load dataset using Hugging Face datasets
imdb = load_dataset('imdb' )
# Examine dataset structure
print(imdb)
# Load tokenizer for BERT
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')

DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

Lab: Tokenization and Dataset Preparation

- Step 4: Prepare tokenization function
- Step 5: Apply tokenization to entire dataset
- Step 6: Prepare training and evaluation datasets
- Transformer tokenization is more sophisticated than classical methods: it includes special tokens ([CLS], [SEP]), padding, truncation, and conversion to token IDs.
- This step prepares the dataset in the exact format expected by the Hugging Face Trainer class.

```
def tokenize_function (examples):
    # Tokenize all texts and truncate to max length of 512
```

```

    return tokenizer(examples['text'], padding='max_length',
truncation=True)

# Apply tokenization to whole dataset
tokenized_imdb = imdb.map(tokenize_function, batched=True)

# Convert to PyTorch tensors and create dataloaders
tokenized_imdb = tokenized_imdb.remove_columns(['text'])
tokenized_imdb = tokenized_imdb.rename_column( 'label','labels' )
tokenized_imdb.set_format( 'torch')

```

Lab: Model Setup and Fine-Tuning

- Step 7: Load pre-trained model
- Step 8: Set up training arguments
- Step 9: Initialize trainer and fine-tune

```

# Load BERT with classification head
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained('bert-base-
uncased', num_labels=2) # Binary classification: positive/negative

```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```

from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(output_dir='./results',
per_device_train_batch_size=4, per_device_eval_batch_size=16,
num_train_epochs=3, weight_decay=0.01, logging_dir='./logs')
trainer = Trainer(model=model, args=training_args,
train_dataset=tokenized_imdb['train'],
eval_dataset=tokenized_imdb['test'])

```

```

# Start fine-tuning
trainer.train()

```

<IPython.core.display.HTML object>

```

-----
-----
KeyboardInterrupt                                Traceback (most recent call
last)
Cell In[8], line 5
      2 trainer = Trainer(model=model, args=training_args,
train_dataset=tokenized_imdb['train'],
eval_dataset=tokenized_imdb['test'])

```

```

822 try:
--> 823     return Variable._execution_engine.run_backward( # Calls
into the C++ engine to run the backward pass
824         t_outputs, *args, **kwargs
825     ) # Calls into the C++ engine to run the backward pass
826 finally:
827     if attach_logging_hooks:

```

KeyboardInterrupt:

It takes too much time so we will test BERT directly on the sentences.

```

# Charger le tokenizer et le modèle pré-entraîné
tokenizer = AutoTokenizer.from_pretrained("textattack/bert-base-
uncased-imdb")
model =
AutoModelForSequenceClassification.from_pretrained("textattack/bert-
base-uncased-imdb")

# Passer en mode évaluation
model.eval()

# Phrase à tester
sentences = [
    "This movie was absolutely fantastic!",
    "I really hated this film. It was boring and too long.",
    "It was okay, not great, not terrible."
]

# Tokenizer les phrases
inputs = tokenizer(sentences, padding=True, truncation=True,
return_tensors="pt")

# Faire les prédictions
with torch.no_grad():
    outputs = model(**inputs)
    predictions = torch.argmax(outputs.logits, dim=-1)

# Afficher les résultats
for sentence, prediction in zip(sentences, predictions):
    label = "Positive" if prediction.item() == 1 else "Negative"
    print(f"Sentence: {sentence}\nPrediction: {label}\n")

Sentence: This movie was absolutely fantastic!
Prediction: Positive

Sentence: I really hated this film. It was boring and too long.
Prediction: Negative

Sentence: It was okay, not great, not terrible.

```

Prediction: Positive

We can use SentenceTransformer which is useful to obtain vector representations of sentences.

```
from sentence_transformers import SentenceTransformer, util

# Charger un modèle BERT optimisé pour les embeddings
model = SentenceTransformer('all-MiniLM-L6-v2')

# Tes phrases
sentences = [
    "This movie was absolutely fantastic!",
    "I really hated this film. It was boring and too long.",
    "It was okay, not great, not terrible."
]

# Obtenir les embeddings
embeddings = model.encode(sentences, convert_to_tensor=True)

# Afficher les vecteurs (optionnel)
for sentence, embedding in zip(sentences, embeddings):
    print(f"Sentence: {sentence}\nEmbedding shape: {embedding.shape}\n")

Sentence: This movie was absolutely fantastic!
Embedding shape: torch.Size([384])

Sentence: I really hated this film. It was boring and too long.
Embedding shape: torch.Size([384])

Sentence: It was okay, not great, not terrible.
Embedding shape: torch.Size([384])

cos_sim = util.pytorch_cos_sim(embeddings[0], embeddings[1])
print("Similarity:", cos_sim.item())

Similarity: 0.46199214458465576
```

Lab: Evaluation and Analysis

- Step 10: Evaluate model performance
- Step 11: Make predictions on specific examples

This doesn't work since we didn't have the time to train the model.

```
# Evaluate on test set
results = trainer.evaluate()
print(f"Evaluation results: {results}")
```

```

from torch.nn.functional import softmax

def predict_sentiment(text) :
    inputs = tokenizer(text, return_tensors='pt', truncation=True,
padding=True)
    outputs = model(**inputs)
    probs = softmax(outputs.logits, dim=1)
    return {'negative': probs[0][0].item(), 'positive': probs [0]
[1].item()}

# Try examples with subtle sentiment or context
examples = [
    "This movie wasn't bad at all. I enjoyed it."
    "The film is predictable but still worth watching."
]

for ex in examples:
    print(f"Text: {ex}")
    print(f"Prediction: {predict_sentiment(ex)}")

```

```

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[21], line 2
      1 # Evaluate on test set
----> 2 results = trainer.evaluate()
      3 print(f"Evaluation results: {results}")
      5 from torch.nn.functional import softmax

```

NameError: name 'trainer' is not defined

Lab: Understanding Attention Weights (Optional)

- Step 14: Visualize attention patterns (if time allows)

```

from bertviz import model_view
from transformers import BertTokenizer, BertForSequenceClassification

# Load model with attention output enabled
model_for_viz = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2,
    output_attentions=True
)

model_for_viz.load_state_dict(model.state_dict()) # Reuse fine-tuned
weights

# Prepare example for visualization

```

```
example = "This film wasn't actually that bad."
inputs = tokenizer(example, return_tensors='pt')
outputs = model_for_viz(**inputs)
```

```
# Visualize attention weights
```

```
attention = outputs.attentions
tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])
model_view(attention, tokens)
```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
-----
-----
RuntimeError                                Traceback (most recent call
last)
Cell In[13], line 11
      4 # Load model with attention output enabled
      5 model_for_viz = BertForSequenceClassification.from_pretrained(
      6     'bert-base-uncased',
      7     num_labels=2,
      8     output_attentions=True
      9 )
--> 11 model_for_viz.load_state_dict(model.state_dict()) # Reuse
fine-tuned weights
     13 # Prepare example for visualization
     14 example = "This film wasn't actually that bad."
```

File

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/
site-packages/torch/nn/modules/module.py:2581, in
Module.load_state_dict(self, state_dict, strict, assign)
    2573         error_msgs.insert(
    2574             0,
    2575             "Missing key(s) in state_dict: {}".format(
    2576                 ", ".join(f'"{k}"' for k in missing_keys)
    2577         ),
    2578     )
    2580 if len(error_msgs) > 0:
-> 2581     raise RuntimeError(
    2582         "Error(s) in loading state_dict for {}: \n\t{}".format(
    2583             self.__class__.__name__, "\n\t".join(error_msgs)
    2584         )
    2585     )
    2586 return _IncompatibleKeys(missing_keys, unexpected_keys)
```

RuntimeError: Error(s) in loading state_dict for

```
"0.auto_model.encoder.layer.5.attention.output.LayerNorm.weight",
"0.auto_model.encoder.layer.5.attention.output.LayerNorm.bias",
"0.auto_model.encoder.layer.5.intermediate.dense.weight",
"0.auto_model.encoder.layer.5.intermediate.dense.bias",
"0.auto_model.encoder.layer.5.output.dense.weight",
"0.auto_model.encoder.layer.5.output.dense.bias",
"0.auto_model.encoder.layer.5.output.LayerNorm.weight",
"0.auto_model.encoder.layer.5.output.LayerNorm.bias",
"0.auto_model.pooler.dense.weight", "0.auto_model.pooler.dense.bias".
```

Lab: Saving and Loading the Model

- Step 15: Save fine-tuned model
- Step 16: Load saved model for future use

```
# Save model and tokenizer
model_path = './fine_tuned_bert_sentiment'
model.save_pretrained(model_path)
tokenizer.save_pretrained(model_path)

# How to load the model later
from transformers import AutoModelForSequenceClassification,
AutoTokenizer
from torch.nn.functional import softmax

loaded_model =
AutoModelForSequenceClassification.from_pretrained(model_path)
loaded_tokenizer = AutoTokenizer.from_pretrained(model_path)

# Test that it works
def predict_with_loaded(text):
    inputs = loaded_tokenizer(text, return_tensors='pt',
truncation=True, padding=True)
    outputs = loaded_model(**inputs)
    probs = softmax(outputs.logits, dim=1)
    return {'negative': probs[0][0].item(), 'positive':
probs[0][1].item()}

print(predict_with_loaded("I really enjoyed this movie!"))
```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at ./fine_tuned_bert_sentiment and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
{'negative': 0.509280264377594, 'positive': 0.490719735622406}
```

Exercise 1: Basic Prompt Experiments

1. Start with this sample article:

<https://addyo.substack.com/p/vibe-coding-is-not-an-excuse-for>

2. Try different prompting approaches:

- a. Basic: "Summarize the above article."

Response:

The article talks about "vibe coding," where people let AI write code without clear planning. It might seem fast, but the code is often weak, missing things like error checks or safety. The writer says we should still use good coding habits, even when using AI.

- b. More specific: "Summarize the above in 3 sentences."

Response:

"Vibe coding" means using AI to write code from very little input. This can lead to bad code with mistakes and security problems. The author says we need to mix AI help with strong coding skills.

- c. Style control: "Summarize in a neutral tone." or "Summarize as bullet points."

Prompt A: "Summarize in a neutral tone."

Response:

The article discusses "vibe coding," a style of programming driven by intuition rather than planning. It notes that while this method can feel productive, it often results in code that is difficult to maintain or understand. The author recommends maintaining coding standards while encouraging creative expression.

Prompt B: "Summarize as bullet points."

Response:

- "Vibe coding" is using AI to write code with little effort.
- This often creates weak code with bugs or no safety.
- You shouldn't trust AI alone—good coding habits are still important.

3. Compare the outputs:

- What differences do you notice?
- Which prompt produced the most useful summary?
- How did format instructions affect the output?

The basic summary provides a general overview of the article. The 3-sentence version is concise and communicates the core ideas clearly.

The bullet point format presents the information in a clear, structured, and easy-to-read way. Among the formats, the bullet point version is the most effective for quick reading and understanding. Specifying the format in the prompt improves readability and clarity. More detailed prompts lead to better, more useful summaries.

Exercise 2: Few-shot learning

1. Try a language translation task:

Zero-shot:

"Translate the following to French: 'I am happy to meet you.'"

Output:

"Je suis heureux de vous rencontrer."

Few-shot:

"English: 'Hello' French: 'Bonjour'"

"English: 'Thank you' French: 'Merci'"

English: 'I am happy to meet you.' French:

Output:

"Je suis heureux de vous rencontrer."

2. Try a style transformation task:

Convert casual text to formal business language

First without examples, then with 2-3 examples

- Zero-shot Prompt:

"Hey, just wanted to let you know the meeting's off."

Output:

"I wanted to inform you that the meeting has been canceled."

- Few-shot Prompt (with 2 examples):

Casual: "Gotta run this by you real quick."

Formal: "I would like to briefly discuss this with you."

Casual: "Can you shoot this over when you get a sec?"

Formal: "Please send this at your earliest convenience."

Casual: "Hey, just wanted to let you know the meeting's off."

Formal:

Output:

"I wanted to inform you that the meeting has been canceled."

3. *Discuss when few-shot seems most beneficial*

Few-shot learning is most beneficial when the task involves ambiguous instructions or requires specific formatting, style, or domain knowledge.

It provides context and examples that guide the model to follow patterns more accurately and consistently.

Few-shot is especially helpful for tasks like tone transformation, domain-specific generation, or structured output where zero-shot may be too generic or unpredictable.

Exercise 3: Creative Generation & Temperature

1. *Use this creative writing prompt:*

"Write a short story beginning with: 'The abandoned spaceship contained something unexpected...'"

2. *Generate completions with different temperature settings:*

- *Low temperature (0.2) - more predictable*

Output:

"The abandoned spaceship contained something unexpected: a small, flickering light in the control room. As Captain Mira approached, she realized it was a distress beacon, still transmitting. Nearby, a journal lay open, its pages filled with coordinates and personal logs. The last entry read, "They told us it was a routine survey mission, but something followed us back..."

- *Medium temperature (0.5) - balanced*

Output:

"The abandoned spaceship contained something unexpected: a garden. Vines crept along the walls, and bioluminescent flowers pulsed gently with light. In the center floated a single orb, humming softly. When Lieutenant Kora reached out, it spoke in a child's voice, "I've been waiting for someone to play with." The ship's systems, though dormant, still responded to the orb's presence."

- *High temperature (0.8) - more random/creative*

Output:

"The abandoned spaceship contained something unexpected: laughter. Not echoes — actual laughter, crystalline and mechanical, coming from a rusted vending machine in the galley. Its screen flickered to life and displayed a smiling octopus in a top hat. "WELCOME BACK, COMMANDER ZOG," it intoned. A small, levitating

pancake floated past the corridor. Lieutenant Vee blinked twice. "I think the ship's AI lost its mind," she muttered, as the walls began to purr"

3. *Compare and analyze:*

- *How does the creativity level change?*

As the temperature increases, the creativity of the generated text also increases. Lower temperatures produce more predictable and conventional outputs, while higher temperatures lead to more imaginative, surprising, and original content.

- *At what point does text become incoherent?*

Text begins to lose coherence at higher temperatures, typically around 0.8 and above. At this level, the output may become less logical or structured, and while it can be highly creative, it sometimes sacrifices clarity or consistency.

- *Which setting would be best for different applications?*

A low temperature (0.2) is best for applications that require accuracy, clarity, and reliability, such as technical writing, summaries, or traditional plot development.

A medium temperature (0.5) offers a good balance of creativity and coherence, making it ideal for storytelling, screenwriting, or general creative writing.

A high temperature (0.8) is most suitable for tasks that benefit from novelty and artistic expression, such as poetry, brainstorming, or experimental literature.

Exercise 4: Role-Based Prompting

1. *Select a simple helpdesk scenario:*

"A user asks: 'I cannot reset my password.'"

2. *Try different role prompts:*

- a. "You are a technical IT specialist. How would you respond?"*

Response:

"Hi, thanks for reaching out. Can you clarify what happens when you try to reset your password? For example, are you not receiving the reset email, is the reset link not working, or is there an error message? Please also let me know which system or application you're trying to access so I can look into the issue more precisely. In the meantime, ensure you're using the correct username or email associated with the account."

- b. *"You are a friendly customer service representative. How would you respond?"*

Response:

"Hi there! I'm so sorry you're having trouble resetting your password—that can definitely be frustrating. I'd love to help you get this sorted out! Could you let me know what happens when you try to reset it? For instance, are you seeing any error messages or not getting the reset email? I'll do my best to walk you through the next steps."

- c. *"You are a cybersecurity expert concerned with verification. How would you respond?"*

Response:

"Hello. For security reasons, I'll need to verify your identity before assisting with a password reset. Could you please provide your employee ID or the last four digits of the phone number linked to your account? Also, let me know what system you're trying to access. Once verified, I'll guide you through a secure reset process"

3. *Compare how the persona affects:*
- *Tone and language*

The technical IT specialist uses a direct and formal tone, focusing on system behavior rather than user emotion.

The customer service representative adopts a friendly and empathetic tone to reassure the user.

The cybersecurity expert maintains a serious and professional tone, emphasizing security over comfort.

- *Depth of technical details*

The technical specialist provides specific troubleshooting steps and assumes some technical understanding from the user.

The customer service representative avoids technical jargon and explains things in a simple, user-friendly way.

The cybersecurity expert includes security-specific terminology and highlights identity verification procedures.

- *Solution approach and priorities*

The IT specialist prioritizes diagnosing the technical issue efficiently.

The customer service representative focuses on keeping the user calm and providing clear step-by-step help.

The cybersecurity expert prioritizes securing the user's identity before proceeding with any reset actions.

Exercise 5: Advanced Techniques

1. Chain-of-Thought:

"A shirt costs \$25. Pants cost \$10 more than the shirt. A customer buys 2 shirts and 3 pants. Calculate the total cost, reasoning step-by-step."

Response:

- Cost of one shirt = \$25
- Pants cost \$10 more than the shirt: $\$25 + \$10 = \$35$ per pant
- The customer buys 2 shirts: $2 \times \$25 = \50
- The customer buys 3 pants: $3 \times \$35 = \105
- Total cost = Cost of shirts + Cost of pants: $\$50 + \$105 = \$155$
- Answer: \$155

2. Format Specification:

"List the capital cities of 5 European countries in JSON format, where each entry has 'country' and 'capital' fields."

Response:

```
[ { "country": "France", "capital": "Paris" },  
  
  { "country": "Germany", "capital": "Berlin" },  
  
  { "country": "Italy", "capital": "Rome" },  
  
  { "country": "Spain", "capital": "Madrid" },  
  
  { "country": "Sweden", "capital": "Stockholm" } ]
```

3. Output Self-Evaluation:

"Write a paragraph explaining quantum computing. Then evaluate your explanation for accuracy and clarity on a scale of 1-10, justifying your rating."

Response:

Quantum computing is a type of computation that uses quantum bits, or qubits, instead of classical bits. Unlike classical bits, which can be either 0 or 1, qubits can exist in a superposition of states, meaning they can represent 0 and 1 at the same time. Quantum computers also leverage other quantum phenomena like entanglement and interference to perform certain calculations much more efficiently than classical computers. This makes them potentially powerful for tasks such as factoring large numbers, simulating quantum systems, and solving optimization problems.

Self-Evaluation:

Rating: 8.5/10

Justification:

The explanation is accurate and clear for a general audience, effectively introducing key concepts like qubits, superposition, and entanglement. However, it could be improved by including a simple analogy to make the abstract ideas even more accessible.

Exercise 6: Quality Review and Iteration

1. Choose a task from your study domain

Task: Analyze a CSV dataset to summarize key statistics and detect any missing or anomalous values.

2. Design an initial prompt

Initial Prompt:

"Analyze this dataset and tell me what you find"

3. Evaluate the output for:

- *Relevance to your goal*

Partially relevant. It gives a general overview but lacks depth.

- *Completeness*

Incomplete. It doesn't specify which columns have missing data or anomalies.

- *Accuracy (if applicable)*

Accurate, but too superficial to assess fully.

- *Style and formatting*

Basic. Lacks structured formatting (like tables or bullet points) and doesn't break down information well.

4. *Revise your prompt to address any issues*

"Please analyze the attached CSV dataset. Provide the following:

- Summary statistics (mean, median, std) for all numerical columns
- Count and percentage of missing values per column
- Detection of any outliers (using IQR or z-score method)
- Output should be structured with headings and tables where appropriate"

5. *Document your prompt improvements and what you learned*

The initial prompt was too general and did not specify what kind of analysis was expected. As a result, the output was superficial and lacked detail. To improve it, I added specific instructions asking for:

- Summary statistics (mean, median, standard deviation) for all numerical columns
- Count and percentage of missing values for each column
- Detection of outliers using the IQR or z-score method
- Structured formatting with headings and tables

These changes made the prompt clearer, more goal-oriented, and ensured the output would be more detailed and well-organized.

I learned that a well-defined and specific prompt leads to much better results. By clearly stating what I expect from the analysis and how I want the information presented, the model is able to generate more relevant, complete, and readable outputs. Good prompt design is essential for getting accurate and useful responses.

Implementing a RAG System

Dataset:

Use one of the following options as your knowledge base:

- A collection of Wikipedia articles on a specific topic of your choice
- Technical documentation from an open-source project
- A set of news articles on a particular subject → **CLIMATE CHANGE**

<https://www.noaa.gov/education/resource-collections/climate/climate-change-impacts>

<https://www.nrdc.org/stories/what-are-effects-climate-change>

<https://www.nationalgeographic.com/environment/article/global-warming-effects>

Implementation Requirements

1. Document Processing and Indexing
 - Process documents into suitable chunks for retrieval
 - Generate embeddings for each chunk using an appropriate model
 - Create a simple vector index for similarity search

```
import requests
from bs4 import BeautifulSoup

# Function to fetch and extract paragraph text from a webpage
def fetch_article_text(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')

    # Find all paragraphs on the page
    paragraphs = soup.find_all('p')

    # Join them into one big string
    text = ' '.join([para.get_text() for para in paragraphs])
    return text

# Article URLs (your 3 sources)
urls = [
    "https://www.noaa.gov/education/resource-collections/climate/climate-change-impacts",
    "https://www.nrdc.org/stories/what-are-effects-climate-change",
    "https://www.nationalgeographic.com/environment/article/global-warming-effects"
]

# Extract the text of all articles
documents = [fetch_article_text(url) for url in urls]
```

This Python code uses web scraping techniques to collect text data from multiple online articles that we picked before on the web.

We chose the theme of climate change because we thought we would have multiple resources to make this lab .

The code employs the requests library to send HTTP requests and retrieve webpage content, and **BeautifulSoup** from the bs4 library to parse and navigate the HTML structure of the pages. Specifically, the “fetch_article_text” function extracts all paragraph (<p>) elements from a given URL and combines their text into a single string.

```
# Function to split large text into smaller overlapping chunks
def chunk_text(text, max_length=500):
    sentences = text.split('. ')
    chunks = []
    current_chunk = ''

    for sentence in sentences:
        if len(current_chunk) + len(sentence) < max_length:
            current_chunk += sentence + '. '
        else:
            chunks.append(current_chunk.strip())
            current_chunk = sentence + '. '

    # Add the last chunk if it's not empty
    if current_chunk:
        chunks.append(current_chunk.strip())

    return chunks

# Split all articles into small chunks for retrieval
all_chunks = []
for doc in documents:
    all_chunks.extend(chunk_text(doc))
```

This code processes large blocks of text by breaking them into smaller, overlapping chunks to facilitate handling, such as for natural language processing tasks.

The “chunk_text” function splits input text into sentences, then groups these into chunks that do not exceed 500 characters (default maximum length).

It maintains sentence boundaries for coherence and appends any remaining text as the final chunk. The second part of the code applies this function to all previously extracted article texts , aggregating the resulting chunks into the all_chunks list.

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

# Load a pre-trained model for turning text into vector embeddings
model = SentenceTransformer('all-MiniLM-L6-v2')
```

```

# Generate vector representations of each chunk
embeddings = model.encode(all_chunks)

# Create a FAISS index to store and search embeddings
dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(embeddings))

c:\Users\pc-portable.net\anaconda3\Lib\site-packages\torch\utils\
_pytree.py:185: FutureWarning: optree is installed but the version is
too old to support PyTorch Dynamo in C++ pytree. C++ pytree support is
disabled. Please consider upgrading optree using `python3 -m pip
install --upgrade 'optree>=0.13.0'`.
  warnings.warn(
c:\Users\pc-portable.net\anaconda3\Lib\site-packages\pandas\core\
arrays\masked.py:60: UserWarning: Pandas requires version '1.3.6' or
newer of 'bottleneck' (version '1.3.5' currently installed).
  from pandas.core import (

```

This code uses **semantic embedding** and **vector search techniques** to prepare text data for efficient similarity-based retrieval. It begins by loading a pre-trained SentenceTransformer model, which converts each text chunk into a high-dimensional numerical vector (embedding) that captures its semantic meaning.

These embeddings are then stored in a FAISS index.

1. Retrieval Component

- Implement a function to convert user queries into embeddings
- Develop a retrieval mechanism to find relevant documents based on similarity
- Include a method to select the top-k most relevant passages

```

# Given a user query, find the top-k most similar chunks
def retrieve(query, top_k=3):
    query_embedding = model.encode([query])

    # Search the FAISS index
    distances, indices = index.search(np.array(query_embedding),
top_k)

    # Return the most relevant chunks
    return [all_chunks[i] for i in indices[0]]

```

This code defines a retrieval function that uses **semantic similarity** to find the most relevant text chunks in response to a user query.

The “retrieve” function first converts the input query into a vector embedding using the same SentenceTransformer model used for the document chunks.

It then searches the FAISS index for the top k most similar embeddings based on vector distance (default is 3).

The function returns the corresponding text chunks from all_chunks that best match the query in meaning.

1. Generation with Context

- Create a prompt template that incorporates retrieved passages
- Use an accessible language model (e.g., Gemini via API, or a local model like Llama-2)
- Generate answers that reference the retrieved information

```
from transformers import pipeline

# Load a small open-source text generation model (e.g., GPT-2)
generator = pipeline("text-generation", model="gpt2")

# Given a query and retrieved context, generate an answer
def generate_answer(query, context_chunks):
    context = '\n'.join(context_chunks)

    # Prompt template that combines context with the query
    prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"

    # Generate text using the model
    response = generator(prompt, max_length=200,
num_return_sequences=1)

    # Extract the generated answer from the model output
    return response[0]['generated_text'].split("Answer: ")[-1].strip()
```

Device set to use cpu

This code uses a pre-trained language model to generate answers based on a user's query and relevant contextual information.

It loads a text generation pipeline using GPT-2 transformers library.

The "generate_answer" function constructs a prompt by combining the retrieved context (a few relevant text chunks) with the user's question.

This prompt is fed into the generator, which produces a natural language response. The output is then parsed to extract and return just the answer portion.

1. Evaluation

- Test your system with at least 10 different queries
- Measure performance using appropriate metrics (relevance of retrieved passages, accuracy of generated answers)
- Compare results

```
# Test your RAG system with a sample question
query = "What are the effects of global warming?"

# Step 1: Retrieve top relevant passages from the vector index
retrieved_chunks = retrieve(query)

# Step 2: Generate an answer using the retrieved information
def generate_answer(query, context_chunks):
    context = '\n'.join(context_chunks)
```

```

# Construct the full prompt including retrieved context and
question
prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"

# Generate a response using the language model
response = generator(
    prompt,
    max_new_tokens=100,          # FIX: Only count new tokens
    num_return_sequences=1,
    do_sample=True,             # Optional: adds randomness
    temperature=0.7             # Optional: controls creativity
)

# Extract the generated answer from the output
return response[0]['generated_text'].split("Answer: ")[-1].strip()

```

```

# Show the result
answer = generate_answer(query, retrieved_chunks)

```

```

print(f"Q: {query}\nA: {answer}")

```

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: What are the effects of global warming?

A: The Earth will experience the effects of climate change as a consequence of a change in the greenhouse gas level. The climate change effects of global warming will be much more severe. A warmer planet will have fewer hurricanes, wildfires, and other violent storms that are more frequent in the coming decades. The most common causes of these hurricanes and wildfires will be caused by greenhouse gases, not from human activities. We will see extreme heat waves, droughts, storms, and extreme drought in

```

test_queries = [
    "What are the effects of global warming?",
    "How does climate change affect weather?",
    "What happens to human health as climate changes?",
    "What are some impacts on agriculture?",
    "How does climate change affect the oceans?",
    "What are long-term climate change consequences?",
    "What are extreme weather events caused by global warming?",
    "How does climate change affect animals?",
    "What are economic effects of climate change?",
    "Can climate change impact water supply?"
]

for query in test_queries:
    print(f"\nQ: {query}")

```

```

retrieved = retrieve(query)
print("→ Retrieved Passages:")
for i, chunk in enumerate(retrieved):
    print(f" [{i+1}] {chunk[:200]}...") # Print first 200
characters
print("→ Answer:", generate_answer(query, retrieved))

```

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: What are the effects of global warming?

→ Retrieved Passages:

[1] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect warmer temp...

[2] The results of this review is synthesized in regularly published reports known as the Intergovernmental Panel on Climate Change (IPCC) reports. A comprehensive report outlines how disruptive a global...

[3] All of these changes are emerging as humans continue to add heat-trapping greenhouse gases, like carbon dioxide and methane, to the atmosphere. When fossil fuel emissions are pumped into the atmospher...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: The primary effect of global warming is to increase the amount of warm air that is absorbed by the atmosphere. This increase in this air will have a dramatic and long-lasting negative impact on the climate.

The average increase in temperature over the past 30 years by a factor of two is a result of a cumulative increase in greenhouse gases. The cumulative decrease in global warming is estimated to be about 1.3 degrees Celsius per decade to 2100. This is equivalent to about 1.4 degrees

Q: How does climate change affect weather?

→ Retrieved Passages:

[1] Global warming and climate change are often used interchangeably as synonyms, but scientists prefer to use "climate change" when describing the complex shifts now affecting our planet's weather and cl...

[2] This increase in moisture in the atmosphere can produce flash floods, more destructive hurricanes, and even paradoxically, stronger snow storms. The world's leading scientists regularly gather to rev...

[3] This warming reduces critical ice habitat and it disrupts the flow of the jet stream, creating more unpredictable weather patterns

around the globe. (Learn more about the jet stream.) A warmer planet...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: The warming of the atmosphere is a major cause of the warming of the Earth. In fact, according to a study published a decade ago, the amount of water vapor in the atmosphere is about the same as that in the atmosphere. The amount of water vapor in the atmosphere is equivalent to two parts of a large brick being blown a million times.

According to this study, the amount of water vapor in the atmosphere is equivalent to a man-made increase in the temperature of the Earth

Q: What happens to human health as climate changes?

→ Retrieved Passages:

[1] All of these changes are emerging as humans continue to add heat-trapping greenhouse gases, like carbon dioxide and methane, to the atmosphere. When fossil fuel emissions are pumped into the atmosphere...

[2] Global warming and climate change are often used interchangeably as synonyms, but scientists prefer to use "climate change" when describing the complex shifts now affecting our planet's weather and climate...

[3] The results of this review is synthesized in regularly published reports known as the Intergovernmental Panel on Climate Change (IPCC) reports. A comprehensive report outlines how disruptive a global...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: Our health depends on the changes in our planet. The climate and the Earth's climate cannot be kept in constant motion. In fact, the climate may change in a way that threatens our health and our health. If that happens, we are at risk of being at risk of long-term health problems.

The most obvious example is the death of a human being as a result of exposure to a greenhouse gas. The death of a human being can be prevented with prevention strategies

Q: What are some impacts on agriculture?

→ Retrieved Passages:

[1] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect warmer temperatures...

[2] Trees, oceans, wetlands, and other ecosystems help absorb excess carbon—but when they're lost, so too is their potential to fight

climate change. Ultimately, we'll need to adapt to warming temperatur...

[3] This type of technology relies on manipulating the Earth's atmosphere to physically block the warming rays of the sun or by sucking carbon dioxide straight out of the sky. Restoring nature may also he...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: A new study by the Center for Biological Diversity shows that the amount of methane in the atmosphere is about the same as that of a typical U.S. chicken. The amount of methane in the atmosphere and the amount of carbon dioxide in the air are not the same—only about 1 percent of the amount of carbon dioxide is captured. So, it seems that the amount of methane in the atmosphere could be similar to what happens in the global climate.

Question: If this

Q: How does climate change affect the oceans?

→ Retrieved Passages:

[1] Trees, oceans, wetlands, and other ecosystems help absorb excess carbon—but when they're lost, so too is their potential to fight climate change. Ultimately, we'll need to adapt to warming temperatur...

[2] Global warming and climate change are often used interchangeably as synonyms, but scientists prefer to use “climate change” when describing the complex shifts now affecting our planet’s weather and cl...

[3] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect warmer temp...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: The same changes that are occurring in the atmosphere—such as ice sheets melting and melting rapidly, rising sea levels, and the rate of ocean acidification—are happening in the ocean. The climate makes a major contribution to the changing environment. The climate also changes the way we consume land, water, and energy. Climate change also affects the way we consume food and water. The Earth is changing rapidly, and it's already changing much more quickly than we thought.

Figure 6

Q: What are long-term climate change consequences?

→ Retrieved Passages:

[1] The results of this review is synthesized in regularly published

reports known as the Intergovernmental Panel on Climate Change (IPCC) reports. A comprehensive report outlines how disruptive a global...

[2] Global warming and climate change are often used interchangeably as synonyms, but scientists prefer to use "climate change" when describing the complex shifts now affecting our planet's weather and cl...

[3] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect warmer temp...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: It is important to understand that global warming is not inevitable. It is the result of a complex process that will occur over much of the future. These impacts will include changes in weather patterns, precipitation, and ice sheets, as well as changes in the climate and the ocean circulation.

An important aspect of climate change is that it is likely to be a significant contributor to climate change. The impacts of global warming are likely to be very large. It is expected to be much larger

Q: What are extreme weather events caused by global warming?

→ Retrieved Passages:

[1] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect warmer temp...

[2] This increase in moisture in the atmosphere can produce flash floods, more destructive hurricanes, and even paradoxically, stronger snow storms. The world's leading scientists regularly gather to rev...

[3] This warming reduces critical ice habitat and it disrupts the flow of the jet stream, creating more unpredictable weather patterns around the globe. (Learn more about the jet stream.) A warmer planet...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: Extreme weather events are caused by climate change. It's no coincidence that global warming has been a major driver of the global economic and environmental crises since the Industrial Revolution.

For example, in 2007, the U.S. Federal Reserve Bank of St. Louis (Fed) and the Federal Reserve Board of Governors (Fed) released a report that examined the impact of climate change on the world's energy infrastructure. The report noted that global warming caused energy infrastructure to decrease in the United States by \$

Q: How does climate change affect animals?

→ Retrieved Passages:

[1] Global warming and climate change are often used interchangeably as synonyms, but scientists prefer to use “climate change” when describing the complex shifts now affecting our planet’s weather and cl...

[2] Trees, oceans, wetlands, and other ecosystems help absorb excess carbon—but when they're lost, so too is their potential to fight climate change. Ultimately, we'll need to adapt to warming temperatur...

[3] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect warmer temp...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: Most animals around the world are suffering from climate change. In fact, the majority of animals around the world are doing better than average on a daily basis. The most important changes are in the animals themselves, but also in the natural environment. Cats are suffering, too, and the human population is falling. In fact, some of the population centers of the world are losing their populations.

So what's a cat?

Cats are more affected than humans by global warming

Q: What are economic effects of climate change?

→ Retrieved Passages:

[1] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect warmer temp...

[2] The results of this review is synthesized in regularly published reports known as the Intergovernmental Panel on Climate Change (IPCC) reports. A comprehensive report outlines how disruptive a global...

[3] Trees, oceans, wetlands, and other ecosystems help absorb excess carbon—but when they're lost, so too is their potential to fight climate change. Ultimately, we'll need to adapt to warming temperatur...

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

→ Answer: Climate change, as discussed in more detail below, can have a significant economic impact. The US government, for instance, spent \$2 billion on climate science research in 2011, and that money has been used to build a new research facility in the Arctic that could

have a ripple effect on the Arctic's climate.

As you can see, these economic benefits are significant. The US government spent a staggering \$2 billion on climate research in 2011, and that money has been used

Q: Can climate change impact water supply?

→ Retrieved Passages:

[1] Trees, oceans, wetlands, and other ecosystems help absorb excess carbon—but when they're lost, so too is their potential to fight climate change. Ultimately, we'll need to adapt to warming temperatur...

[2] Sources of this gas include agriculture (mostly livestock), leaks from oil and gas production, and waste from landfills. One of the most concerning impacts of global warming is the effect warmer temp...

[3] Global warming and climate change are often used interchangeably as synonyms, but scientists prefer to use “climate change” when describing the complex shifts now affecting our planet’s weather and cl...

→ Answer: The global surface temperature rise between the pre-industrial and post-industrial period has been increasing slowly, but it has been decreasing at a faster rate. This is due to increased water availability due to the changing weather conditions. During the first half of the 20th century, there was an increase in precipitation and snowfall. This increase increased the surface temperature, but it was slower in the subsequent decades. The rise in precipitation and snowfall resulted in an increase in the amount of

This code tests the **Retrieval-Augmented Generation (RAG)** system on a set of climate-related questions.

It defines a list of test_queries covering various aspects of climate change, such as its effects on weather, health, agriculture, and the economy. For each query, it retrieves the most relevant text chunks using **semantic search** and displays them.

It then generates a context-aware answer using the “generate_answer” function, which combines the retrieved passages with the query to produce a response via a language model.

```
def baseline_answer(query):  
    prompt = f"Question: {query}\nAnswer:"  
    response = generator(prompt, max_new_tokens=100)  
    return response[0]['generated_text'].split("Answer: ")[-1].strip()  
for query in test_queries:  
    rag = generate_answer(query, retrieve(query))  
    baseline = baseline_answer(query)  
    print(f"\nQ: {query}")  
    print("→ Baseline Answer:", baseline)  
    print("→ RAG Answer      :", rag)
```

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: What are the effects of global warming?

→ Baseline Answer: Global warming is an effect on ecosystems and human health. However, it is unclear how much global warming actually will affect the food supply in the United States, or in other parts of the world. The United States is projected to lose a significant share of its food security, including significant portion on energy products and services. An average of 20% of the United States population lives in developing countries that are at very high risk of food insecurity. The resulting scarcity of food leads to large-scale food insecurity and

→ RAG Answer : Although the IPCC is not yet able to definitively say exactly how much the climate will change, it predicts that it will increase or decrease. This is because climate change changes the way the Earth's climate affects other things like rainfall and temperatures, and is often thought to be an environmental disaster. The most important impact of global warming is an increase in sea surface temperatures of about 1.7°C, which is about one degree Celsius above pre-industrial levels. This is a huge

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: How does climate change affect weather?

→ Baseline Answer: The weather in South Africa is far darker than the other six countries in the region. The weather is colder than in the continental US.

Source: Weather.net

→ RAG Answer : Climate change is changing the climate.

Scientists, especially those from the scientific community, tend to focus on two parts of the picture: the effects of global warming on snow and ice cover and the effects of climate change on precipitation and precipitation patterns. These two aspects can be viewed separately from each other in the context of climate change.

The first part of the picture is much more complex. The climate change caused by climate change has been known for decades, but it has not been

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.
Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: What happens to human health as climate changes?

→ Baseline Answer: We are not living with the fact that climate change poses a real threat to human health. The average person is sick and vulnerable to diseases from a variety of possible causes including:

Infertility in particular

Pregnancy and Infertility

Kidney disorders

Cancer in particular

Obesity in particular

Inflammation

Oral antibiotics

Hepatitis (Teflonil, P. pneumoniae)

Heart disease

Certain cancers

→ RAG Answer : The world continues to experience significant health impacts—particularly cardiovascular disease, diabetes, cancer, respiratory illnesses, and chronic obstructive pulmonary disease (COPD)—from human activities such as fossil fuel burning. These changes are often overlooked in the media and political discourse. These changes are often ignored by the public and the media's focus on the health impacts of the fossil fuel industries.

This review is a reflection of recent research and a critical perspective on the health and environmental consequences of climate change

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.
Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: What are some impacts on agriculture?

→ Baseline Answer: The Agricultural Produce Act allows farmers to set farm land using "farm-friendly" methods.

1. Farm plants, for example, must be grass-fed. 2. Livestock must have been grown in a clean and fertile environment. 3. The farm has to be open to all natural light. 4. All farm animals are registered with a minimum standard 1 year of training in agriculture.

Farm plants and animals are inspected every 3 to 4 years.

The Act also protects animals

→ RAG Answer : There are certainly many effects on agriculture, but one of the most significant is the impact of climate change. In the 1990s, the World Bank estimated that farming could contribute to 4.3 billion tons of greenhouse gases, or about 20% of the world's emissions. It's estimated that if we cut back to only 1%, the world's greenhouse gas emissions would amount to around 9 billion tons. That's equivalent to just under 2% of the world's greenhouse gas emissions.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: How does climate change affect the oceans?

→ Baseline Answer: This question is similar to the one in which you asked the question, namely, how does climate change affect the oceans? I am going to try to bring this down to the "global warming".

I know many people with a deep understanding on oceans will disagree, but it seems to me that this is not what the IPCC are proposing, so we should not expect the "global warming" statement to be quite that clear.

One must also realize that in addition to the CO2 that

→ RAG Answer : This is a complex issue. Global warming is increasing the amounts of water that are trapped within the oceans. In the past, the oceans have been less active. Some of these changes are irreversible, and some are not. The Earth's climate is changing rapidly due to the effects of human activity, natural disasters, and other factors. There is a lot of uncertainty about what is causing this change, and the magnitude of the effects is hard to predict. But there are some

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: What are long-term climate change consequences?

→ Baseline Answer: The Climate Model is a concept that states that while temperatures may rise to a certain amount for some periods or for little or no longer there will be "long-term consequences". These will occur as a consequence of changes in the soil and water chemistry

of our climate systems or an increase in carbon pollution as well as of other influences which may affect this. Long-term changes in land surface concentrations of carbon dioxide (CO₂) and other greenhouse gas (GHGs), such as the release of

→ RAG Answer : For the long-term, there are many uncertainties about the long-term effects of climate change. Such uncertainties include:

- How quickly the Arctic will cool, and how quickly it will change over time.

- The temperature of the Arctic Circle.

- The frequency of ice loss in the Arctic Ocean.

- The role of climate change in the increasing rate of sea level rise.

In addition, there are many uncertainties about the future of the Arctic

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: What are extreme weather events caused by global warming?

→ Baseline Answer: Extreme weather events generally cause extreme shifts in a region such that their severity in many ways mirror those of heat or humidity in that region, so that the climate system in general can be adjusted in some way, thereby mitigating these weather extremes. Thus, extreme weather events can be described to reduce the risk that the temperature rises for a given period or period of time. Furthermore, extreme weather events can occur more frequently in countries with high levels of inequality or in regions with low levels of well-being or in

→ RAG Answer : Extreme weather events can be caused by anything from drought to fires to extreme weather events like floods, landslides, and fires. The question is, how do we know what causes these events?

The first thing to understand is how the Earth works. Earth's surface is covered by a layer of ice called the mantle. The ice is covered by the thin, deep layers of water ice that are called "biosphere layers."

The Earth is covered by the mantle in the

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: How does climate change affect animals?

→ Baseline Answer: I believe there are several major processes and interactions that affect the physical and behavioral changes of animals. However I do not accept that the amount of animal growth that has occurred over the past 200,000 years is the single biggest impact. It is not just about growing big or spreading the disease; much larger changes occur over time under these conditions.

Figure 7 shows the total number of species that have multiplied or decreased in size over time by a factor of 2, or approximately 2 billion years or more.

→ RAG Answer : Because they are all mammals or birds. Because they are all mammals or birds. Because they are all mammals or birds. Because they are all mammals or birds. Because they are all mammals or birds.

Because they are all mammals or birds. Because they are all mammals or birds. Because they are all mammals or birds. Because they are all mammals or birds.

How does climate change affect species?

T

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Q: What are economic effects of climate change?

→ Baseline Answer: Many economists have argued that climate change is an event that has a major bearing on aggregate economic activity. These studies find that economic activity declines in response to climate change and that those who experience the declines have a larger income share of that income, thus supporting the notion that economic activity declines. The argument is that a decline in income or job opportunities would tend to cause a bigger decline in overall economic activity.

Another important analysis, conducted at the University of Iowa, concluded: "These data from these two

→ RAG Answer : A major economic cost to humanity is global warming. A significant cost to society is that of global warming, especially in the short term. A major economic cost is that of global warming, particularly in the short term. A major economic cost to society is that of human-induced climate change. A substantial amount of that costs will be borne by the rest of the world.

Source:

Ibid, p. 53.

Source:

K., A

```
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
```

Q: Can climate change impact water supply?

→ Baseline Answer: No such consequences. When water is depleted (in large part due to fossil fuel consumption), it is released by large swaths of the land in the area (by runoff and from agriculture). This runoff, as part of the climate change, will have greater and stronger impacts on water usage than the rest.

What is the impact of water use on sea level rise (and associated water quality and depletion)?

What is the impact of changing climate on water use?

Where can I go to learn

→ RAG Answer : The answer to this question is yes, but the answer is complicated. Water supply is critical to any ecosystem's survival, so it is important to understand how much water we are getting from the atmosphere and how much it's being used to build or store.

In order to understand how much water we are getting, our first question must be: How much water does it take to build something? This is the question that many people have about how much we're getting from the air

This code compares two approaches to answering user queries: a baseline method and a Retrieval-Augmented Generation (RAG) method. The “baseline_answer” function simply feeds the query into the text generation model without any external context, generating a generic answer.

In contrast, the RAG method uses relevant text chunks retrieved via semantic search to provide informed, context-aware answers. For each query in the test_queries list, both methods are run, and their responses are printed for side-by-side comparison.

Analysis of Retrieval Quality and Its Effect on Generation

The retrieval quality in this system plays a pivotal role in shaping the relevance and coherence of the generated answers.

For many queries (like the effects of climate change), the retrieved chunks are topically appropriate and provide specific, fact-based context, which enables the generator to make grounded responses. However, there are clear instances where the retrieval system fails to capture truly relevant content.

For example, some retrieved passages are either too general, partially cut off, or unrelated to the question being asked, such as in the queries about animals or long-term consequences. This inconsistency limits the effectiveness of retrieval-augmented generation (RAG) and can cause the model to generate vague or repetitive responses.

Additionally, because the FAISS index is built from fixed-sized text chunks with no semantic overlap scoring, it's possible for the top-k results to miss more informative passages that would be crucial for generating nuanced questions.

Discussion of Factuality Improvements Compared to Pure Generation

The introduction of retrieval in this pipeline clearly improves factual grounding over baseline GPT-2 generation. The baseline responses often suffer from hallucination, incoherence as seen in the answers about agriculture and human health.

In contrast, the RAG pipeline leverages real passages extracted from reputable climate-related sources, and the generated responses often contain more realistic issue framing and terminology reflective of actual articles. For instance, the RAG response to the question on human health references conditions like COPD and fossil fuel impacts, which are aligned with real-world discussions on the topic.

However, factuality is still not fully guaranteed, as the generator occasionally produces logically inconsistent or speculative conclusions based on poor or ambiguous context. Nevertheless, the RAG architecture represents a substantial improvement in factual consistency compared to prompting the language model without any context, especially when that model is relatively small and unoptimized like GPT-2.

Error Analysis and Potential Improvements

The system currently shows several points of failure that limit the reliability and fluency of the generated answers. One prominent issue is chunking: by splitting the text into fixed-length overlapping segments without regard for sentence or paragraph boundaries, many chunks contain incomplete sentences or ideas.

This is evident in responses that feel repetitive or off-topic. Another issue arises in the embedding stage using a single pass of the SentenceTransformer model without domain adaptation may not capture fine-grained distinctions between related climate topics.

Furthermore, the generation component, powered by GPT-2, is relatively weak and often struggles with coherence, especially under ambiguous prompts or noisy input. The prompt formatting could be improved by making the instruction clearer and separating context more explicitly.

To address these shortcomings, future improvements could include smarter text chunking (in sentences or in paragraphs for example), integrating a semantic search re-ranker for retrieval, fine-tuning or replacing GPT-2 with a more capable model like GPT-3.5. These enhancements would substantially improve both factuality and fluency of the answers generated.